Bridging the Abstraction Gap: A Systematic Approach to Rule-Based Transformational Design for Embedded Systems

FAHIMEH BAHRAMI, EECS, Kungliga Tekniska Hogskolan, Stockholm, Sweden
RODOLFO JORDÃO, Kungliga Tekniska Hogskolan, Stockholm, Sweden
INGO SANDER, Kungliga Tekniska Hogskolan, Stockholm, Sweden
INGEMAR SÖDERQUIST, Business Area Avionics, Saab AB, Linköping, Sweden

Raising the level of abstraction is considered key to addressing the ever-increasing complexity of embedded system design, but it causes additional challenges due to the larger abstraction gap between the initial specification and the final implementation. This article addresses the current lack of systematic design methods by extending existing design-transformation-based approaches and wrapping them into a rule-based transformational design methodology for heterogeneous multi-processor platforms. The methodology cross-fertilizes embedded system design with program transformation techniques while taking into account the interplay of tight constraints and platform heterogeneity inherent in such systems. It advocates step-wise transformations starting from initial requirements to yield a final refined model that is efficient for implementation.

To consider the effect of transformations on different properties of the system at each step, the system is specified with a set of requirements, an application model, a platform model, and a set of mapping decisions; referred to as the *RAMP* view of the system. The RAMP view and its carefully selected underlying unified *abstract graph representation* lay the foundations for mechanizing and potentially automating design transformations. A *pattern matching* technique is introduced and a proof-of-concept tool is implemented that automatically detects all possible transformations by matching the patterns defined by the transformation rules to the abstract graph representation of the system model. The underlying graph representation enables complex transformations on different aspects of the design, resulting in an improved design space definition. The design space can be explored by application-platform co-exploration techniques, yielding the most promising sequence of application transformations alongside the best matching platform. The applicability and potential of the proposed methodology are showcased through the design of both an image processing system and a cloud detection system.

CCS Concepts: • **System design → Embedded systems design**; • **Design methodologies → Transformational design methodologies**; • **Program transformation → Design transformation**; • **Transformations →** *Rule-based transformations;*

Additional Key Words and Phrases: Embedded and cyber-physical systems, formal system design, design transformation

This research was funded by Sweden's Innovation Agency partially via the NFFP7 project 2019-02743: TRANSFORM - Design transformation for correct-by-construction design methodology and the Advanced and innovative digitalization project 2021-02484 EARLY BIRD - Seamless System Design from Concept Phase to Implementation.
Authors' Contact Information: Fahimeh Bahrami, EECS, Kungliga Tekniska Hogskolan, Stockholm, Sweden; e-mail: fahimehb@kth.se; Rodolfo Jordão, Kungliga Tekniska Hogskolan, Stockholm, Sweden; e-mail: jordao@kth.se; Ingo Sander, Kungliga Tekniska Hogskolan, Stockholm, Stockholm, Sweden; e-mail: ingo@kth.se; Ingemar Söderquist, Business Area Avionics, Saab AB, Linköping, Sweden; e-mail: ingemar.soderquist@saabgroup.com.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).
ACM 1539-9087/2026/03-ART20
https://doi.org/10.1145/3714412

ACM Reference Format:
Fahimeh Bahrami, Rodolfo Jordão, Ingo Sander, and Ingemar Söderquist. 2026. Bridging the Abstraction Gap: A Systematic Approach to Rule-Based Transformational Design for Embedded Systems. *ACM Trans. Embedd. Comput. Syst.* 25, 2, Article 20 (March 2026), 33 pages. https://doi.org/10.1145/3714412

1 Introduction

Apart from the tight constraints imposed on embedded systems, the complexity of designing such systems has increased due to several reasons: the ever-increasing demands for new applications, the possibility of using more and more processors on a chip and accelerators, and the growing trend towards distributed embedded systems. Raising the level of abstraction of the initial system model is regarded as a critical technique to manage the complexity of designing today's systems [26]. Although this approach has many potential benefits, it widens the abstraction gap between specification and implementation. Naturally, high-level system specification models are not directly suitable for implementation. A promising technique to systematically and gradually modify, optimize, and refine high-level models for implementation is to use a *model-based transformational approach* as advocated by Sifakis et al. [43]. The use of high-level optimizations is also supported by the arguments of Kahng et al. [24], stating that "high-level optimizations have been shown to have a much more significant impact on power than low-level optimizations".

In a model-based design approach, the design space is determined by the application and the platform models. How the application and platform spaces are modeled limits the design space and, therefore, the implementation. Nevertheless, due to the declarative nature of specifications at higher abstraction levels, the initial system specification model is more suitable for capturing the functionality of the system rather than for implementation. Meanwhile, traditional platform-specific compilation techniques are not enough to make the models suitable for implementation if the platform is not fixed or is re-configurable and the design space is extended with the platform space. Accordingly, it is of great importance to (1) select a suitable modeling paradigm for the system model, and to (2) systematically and successively *transform* and refine the system specification model to an efficiently implementable model [14].

Although there exist earlier attempts to exploit program transformation techniques in model-based system design [38, 41], they mostly address the definition and verification of transformation rules for system models and lack an accompanying transformational design methodology. This article addresses the current lack of *systematic* and *automated* transformational design methods for embedded systems. To fulfill this aim, the following novel concepts and techniques are introduced and partially formulated and developed:

- **Transformational design methodology:** A vision for automation of a transformational design process based on the extensions of the works in [38] and [41] is outlined and wrapped into a systematic design methodology (Section 4). The methodology supports both rule-based semantic preserving and non-semantic preserving transformations [41]. At each transformation step, the design is captured as a system model consisting of the requirements R, the application model A, the mapping model M, and the platform model P, thus, *RAMP*. The transformations on different elements of the tuple $S = (R, A, M, P)$, referred to as the RAMP view of the system, enable high-level transformation-based design space identification and exploration at the early stages of a design process. RAMPs are also used to keep the history of transformation sequences that provide transparent transformations in conjunction with transformation rules (Section 2).
- **Carefully selected underlying application (behavioral) models:** The combination of **models of computation** (**MoCs**) and data parallel skeletons enables new means for

optimizations in our design methodology. That is, transformations can modify different aspects of the application model, for example, task and data parallelization. Also, defining both process constructors and skeletons as **higher-order functions** (**HOFs**) enables consistent syntax and semantics for transformations (Section 2.1).

— **Carefully selected underlying abstract graph representation:** Transformations are applied to the system graph developed by the ForSyDe IO meta-modeling framework proposed in [21], which is based on a formalism of system graphs and trait hierarchies. In order to achieve this, trait hierarchies for synchronous MoC and skeletons are created (Section 4.2.1). The graph representation of the RAMP view of the system gives many advantages, including (1) providing a single common ground for transformations on different aspects of the design, (2) eliminating the integration efforts, and more importantly (3) providing support for complex transformations (Section 4.2.1).

— **Automatic pattern matching and transformation:** An automatic pattern matching technique is introduced to detect possible transformations. A proof-of-concept tool that demonstrates the potential of the technique is developed. The framework matches the patterns defined in the transformation rules to the abstract system graph and identifies all potential transformations at each transformation step (Section 4.2.1). In addition, a foundation for automatic transformations is established and a concrete example of automatic application of a specific transformation rule is provided (Section 4.2.2).

— **Case Studies:** We use an illustrative example to clarify the key concepts and techniques introduced in this work. The applicability and potential of the proposed methodology are demonstrated through the design of an image processing system and a cloud detection system. We evaluate the effectiveness of our pattern matching tool for both systems and assess the impact of the applied transformations by feeding the transformed graphs into the DSE tool from [23], followed by an analysis of the resulting performance metrics (Section 5).

The scope of the paper and its main objectives are (1) to provide a vision of a systematic and semi-automated rule-based transformational design flow for embedded systems where rule-based program transformation techniques can be exploited, (2) to take the first steps toward automation of the flow, and (3) to demonstrate the potential of the proposed techniques.

2 System Specification Models and Prerequisites

A system is modeled with four different *views*, each highlighting one aspect of the design: *the set of requirements R*, *the application model A*, *the platform model P*, and *the mapping model M*. *We express the system model at each transformation step as a tuple* $S = (R, A, M, P)$ *that we refer to as a RAMP*. RAMPs imply slopes that connect the system specification model on the highest abstraction level to the final implementation. At the highest abstraction level, only requirements are defined (i.e., $S = (R, \emptyset, \emptyset, \emptyset)$), and at the lower levels, all elements of the RAMP are partially or completely defined (i.e., $S = (R, A, M, P)$).

The requirement view of the system (R) in RAMPs emphasizes *continuous consideration of requirements*. The inclusion of requirements in all design steps ensures that the design process is consistently guided by the requirements and, at each step, decisions are made with a clear reference to them. Drawing inspiration from Gajski's [16] and Kienhuis's [27] Y-chart approaches, our method goes beyond by explicitly refining requirements. In contrast to conventional practices that focus primarily on application, platform, and mapping views, our design process maintains ongoing alignment with requirements, mitigating deviations as the project progresses. This commitment to continuous reference acknowledges the dynamic nature of requirements in today's complex embedded systems, where stakeholder needs, technological advancements, and project priorities can lead to evolving requirements that must be seamlessly integrated and addressed.

Moreover, by employing RAMPs along with their underlying abstract graph representation (Section 4.2.1), we advocate not only an iterative but also an incremental approach to system design. We believe that since new embedded systems need to operate in a complex and evolving environment, they need to be designed using a design approach in which requirements and design alternatives are gradually developed through cooperation between cross-functional collaborators and stakeholders. Such a design approach prescribes frequent integration, incremental changes, and continuous refinement.

In a model-based transformational design process, clear semantics are essential for enabling an efficient, systematic design. In this section, the modeling concepts and prerequisites of different elements of the RAMP view are explained.

2.1 Application Model

In the current work, the application model is a hierarchical *process network* that expresses the concurrent execution of several *processes* and the data communication between them. The properties of such models are characterized by the **tagged signal model** (**TSM**) [29]. In a process network, "a process is a partial function on *signals*" [29], and the processes communicate by signals (i.e., by writing to and reading from signals). Based on TSM, a signal is a collection of *events*, characterized by a *tag* and a *value*. When a process is executed, events are consumed and produced. The process execution follows the semantics of the underlying **model of computation** (**MoC**), for example, synchronous data flow MoC, synchronous MoC. The functionality of processes themselves is arbitrary: it can be defined through mathematical descriptions or algorithms.

In addition to MoCs that specify temporal properties, *parallel skeletons* [8, 44] are used to capture the potential parallelism of an application. *The combination of these enables the definition of transformations that affect different aspects separately.* Both *process constructors* [38] (i.e., used to construct process networks for different MoCs) and skeletons are building blocks of the application model and are defined as HOFs. *HOFs can be exploited to define consistent syntax and semantics for transformations on process constructors and skeletons.* In the sequel, these concepts are elaborated.

2.1.1 Process Constructors. Following [38, 41], a clean separation between computation and communication is achieved by means of the concept of process constructors. A process constructor is an HOF, which takes pure functions (i.e., side-effect-free functions) and values as arguments and constructs a process. The process constructor describes the communication interface of the process, while the arguments to the process constructor describe its computation. This clean separation enables defining powerful transformation rules for processes that have the same process constructors but have different arguments. Few process constructors are needed to define a well-structured model. Table 1 presents some of the basic process constructors.

2.1.2 Skeletons. Due to the domination of multi-processor systems, digital-signal processors, and customized parallel hardware accelerators in many embedded systems, modifying the spatial behaviors that express potential data parallelization is desired. Data-parallel skeletons [47] are used to develop parallel programs while keeping the timing analyzability for real-time embedded systems. They enable the expression of parallel patterns and have been used to model abstract building blocks with a predefined implementation on a parallel machine. Skeletons enable transformations that exploit potential parallelism. The initial model that often captures the most natural way of parallelization is transformed into a model that is more suitable for parallel implementation. However, in our work, the target platform is not necessarily predefined, which makes the application of data parallelization transformations more challenging. On the other hand, this uncertainty unveils a larger design space with more opportunities for exploration and optimization. Table 1 outlines two fundamental skeletons.

2.2 Platform Model

Apart from the clean separation between communication and computation achieved by process constructors, the concept of "separation of concerns" [26] underscores the significance of the distinction between the application model and the platform model. As explained in Section 2.1, the application model defines the functionality of the system independently of the platform. The platform model, on the other hand, is built by instantiating the platform and component templates often available in a module library and composing them in a way that satisfies the instantiation constraints (i.e., constraints that define the valid composition of the components).

The component templates are characterizations of the principal building blocks of the "*common architectures*" that are "architectures that can support a variety of applications" [15]. They are characterized by parameters and control functions and can model different configurations. For embedded systems, at the system level, the "*basic platform*", that is, a platform built of the principal components that "remain the same within a certain degree of parameterization" in different designs [15], is a set of processors and memories that are connected using interconnections. Therefore, the components comprise (1) computational or **processing elements** (**PEs**) like processors or accelerators, (2) **storage units** (**SUs**) such as shared or local memories, (3) **communication elements** (**CEs**) connecting PEs and SUs like busses or routers, and (4) I/O units. The platform model reflects the *design decisions* on component selection and composition in terms of the quantities of different instances of the PEs, SUs, CEs, and I/Os, as well as the communication topology.

2.3 Mapping Model

Because of the clean separation between the application model and the platform model, a relation between them has to be established during the design process. The mapping model captures the application to platform binding, partially or completely. It also reflects the design decisions on scheduling, when necessary. We use the simple but useful design space modeling technique carried out in [6]. The application model and the platform model are represented as graphs. As they are, by construction, bipartite [12], the mapping model consists of edges between them, embodying any possible mapping. This definition implies that the application and platform models must have a *corresponding* graph representation, but do not need to be inherently graph-based models. In other words, the descriptions are not restricted to a particular graph format, such as GraphML. Instead, they can be in any format that allows for bidirectional translation between the process node descriptions and a graph-based representation.

2.4 Requirements

Top-level requirements are decomposed into more detailed ones to be assigned to different subsystems. *Decomposed requirements* should be traceable and need to be verified after each transformation step. They may also be further decomposed in each transformation step. Furthermore, in high-level design, the refinement of the functional abstraction level models to the components abstraction level models (Section 4) introduces additional requirements resulting from design decisions. These *derived requirements* are not directly traceable to higher-level requirements. They are decisions commonly made by an expert engineer, that is, rules of thumb in engineering, that in an automated design process can be expressed by rule-based algorithms describing acceptable decisions, for example, energy rules, security rules, and scheduling rules. Examples of derived requirements are requirements on execution frequency, robust partitioning, redundancy, and computational load. It is important to emphasize that the rigorous handling of requirements, specifically the use of formal methods for requirements verification, falls outside the scope of this article and is left for future work.

3 Design Transformations and Problem Statement

The idea of *design transformation* originates from *program transformations* [34]. The general idea of program transformation is to start from an initial, easy-to-write and easy-to-verify, and probably inefficient program $Prog_0$ and end in an efficiently implementable program $Prog_n$ after a sequence of semantic-preserving program transformations (i.e., $Prog_0 \xrightarrow{T_0} Prog_1 \xrightarrow{T_1} \ldots \xrightarrow{T_{n-2}} Prog_{n-1} \xrightarrow{T_{n-1}} Prog_n$). Therefore, for a given cost function C, the inequality $C(Prog_0) \geqslant C(Prog_n)$ should be guaranteed. We follow the same idea in our transformational design methodology. Hence, starting with initial high-level system model $S_0 = (R_0, A_0, M_0, P_0)$, the methodology advocates transformations to yield the system model $S_n = (R_n, A_n, M_n, P_n)$ that satisfies $C(S_0) \geqslant C(S_n)$. It is also possible that an inefficient version of system model S_i, such that $C(S_{i-1}) < C(S_i)$, is admitted since it may open the door for subsequent transformations leading to a more efficient version of the system model than S_0. However, in contrast to program transformation, the cost function abstracts a broader range of goals when it comes to embedded system design. Often, there are multiple contradictory objectives to be optimized simultaneously, for example, power consumption, performance, and reliability. Moreover, compared to program transformation techniques that only apply semantic preserving transformations, transformations that change the semantics of the model and refine it are essential in a high-level embedded system design approach to yield an efficient implementation. The implications of such transformations are expressed in the transformation rules, and their application requires verification of both local and global properties of the transformed model [37]. In addition, platform heterogeneity inherent in embedded systems makes the application of program transformation techniques in system design more complicated. These arguments give further evidence for the need for a systematic transformational design methodology that mechanizes but not necessarily fully automates the design transformations.

4 Transformational Design Methodology

The operational flow of the proposed methodology is shown in Figures 1 and 3 (in the following, words in *italics* refer to the corresponding part in these figures). The flow comprises two main phases: (1) the specification phase, and (2) the transformation phase (with synthesis being regarded as part of the transformation phase), which belong to at least two abstraction levels: the functional abstraction level and the functions decomposed to the components abstraction level. The central concepts and techniques introduced and developed in this work, as well as the potential and applicability of the proposed transformational design methodology, are illustrated by a running example of a typical image processing system throughout this section. This system converts an RGB image to an ASCII art image using brightness correction and edge detection (Sobel).

4.1 Specification Phase

Figure 1 shows the specification phase of our transformational design flow. The starting point is requirement specification. However, it should be noted that requirements engineering is out of the scope of this article, and we only generally explain how the top-level requirements are refined to the initial system specification models and how they are considered in the rest of the design process. The *top-level requirements* that are usually in text format are *decomposed* into several sub-requirements. The functional and non-functional requirements are further decomposed to be assigned to sub-systems using a *requirement analysis/management tool* [19]. The result of this step is a requirements tree, which is extended and used for tracing the consequences of transformations on requirements during the transformation phase. At the next step of requirement specification, the leaf nodes of the requirement tree are associated with a set of functions by means of a formal functional modeling framework (*Application modeling framework*). These functions are arguments

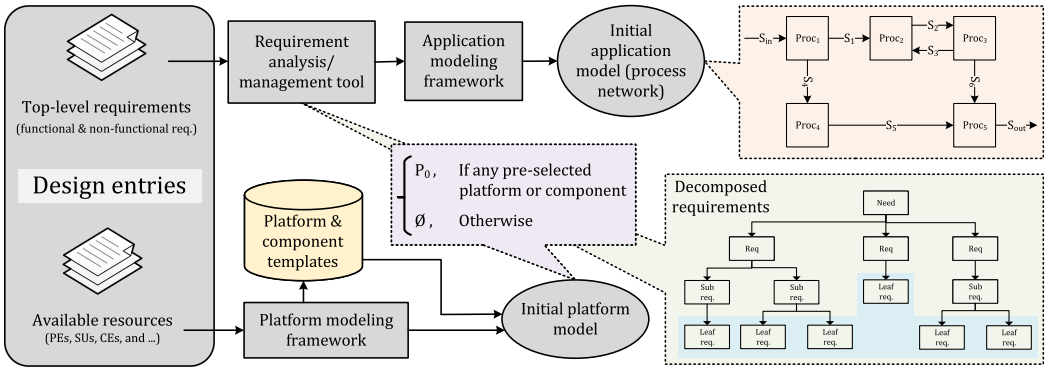


Fig. 1. The operational flow of the specification phase (dashed boxes illustrate sample results of each step).

to HOFs, that is, process constructors and skeletons, and are not modified during the transformation process. We assume that the functions are proven to be functionally correct. One of the outputs of the specification phase is an *initial application model*, which at least specifies all the functional requirements.

In essence, the initial application model is a hierarchical process network constructed using process constructors and skeletons that take functions as arguments. These functions can range from simple logical or arithmetic operations to more complex, larger code segments. When the functions are basic operations, there are greater opportunities for optimization, as this allows us to analyze the processes' contents and even *modify their internal structures*. Transformation rules modify the structure of the process network without altering its functionality. Many studies ([11, 30, 31], and [32]) have demonstrated that transformations that modify the structure of a process network can significantly reduce the total execution time of an application.

Processes in the initial application model are typically coarse-grained, making them well-suited for capturing the system's functionality, though not necessarily optimized for implementation. Various process sizes are explored through these transformations.

As illustrated in Figure 1, another design entry defines the available resources, for example, the reuse of previously used architectures. The platform and component templates are abstracted and derived from the available resources. The templates comprise: (1) the parameterized platforms and components, (2) their instantiation constraints that define the valid composition of the components, and (3) their performance characteristics. If there is any pre-selected platform or component, it should be reflected in the initial platform model; otherwise, it is empty.

Example 4.1 (Initial System Model). We use ForSyDe (Formal System Design) [41] as the application modeling framework. It is suitable to host our transformational design as it supports the modeling concepts explained in Section 2.1. ForSyDe is based on MoCs and the functional programming paradigm. It supports modeling heterogeneous systems by providing a unified, integrated framework for using different MoCs. Figure 2 shows the initial application model based on the synchronous MoC and skeletons captured by the `ForSyDe.Shallow`[1] library . However, we remark that the proposed approach is general and not limited to a particular modeling framework as long as the modeling framework supports MoCs and data parallel skeletons. Since models developed using ForSyDe are executable, the functionality is verified through simulation. Listing 1 models net-list of the application model. Table 1 denotes the ForSyDe notations used in this

[1] https://github.com/forsyde/forsyde-shallow

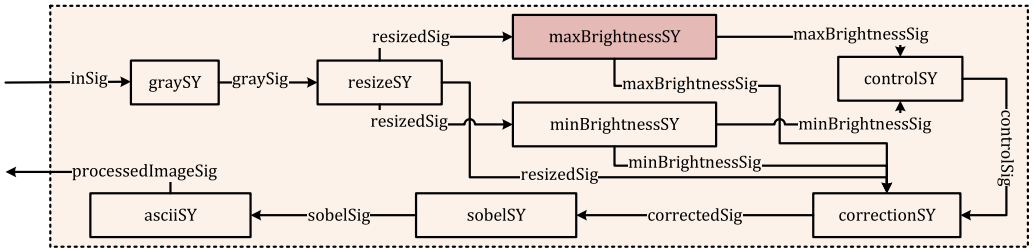


Fig. 2. Image processing initial application model.

Table 1. ForSyDe Notations, Process Constructors, and Skeletons used in the Image Processing Example

Modeling concept	Notation	Definition	Mathematical definition
Signal	s	A sequence of events, where each event e_i has a tag and a value	$s = \{e_0, e_1, \dots\}$
Vector	v	A shallow interpretation of an array of m elements	$v = [v_0, v_1, \dots, v_{m-1}]$
Process constructor	*mapSY*	Constructs a process that applies a function f on each event of a signal	$mapSY(f, \{e_0, e_1, \dots\}) = \{f(e_0), f(e_1), \dots\}$
	zipWithSY	Constructs a process that applies a two-input function f pairwise on two signals	$zipWithSY(f, \{e_0, e_1, \dots\}, \{e'_0, e'_1, \dots\}) = \{f(e_0, e'_0), f(e_1, e'_1), \dots\}$
	zipxSY	Constructs a process that converts a vector of signals into a signal of vectors	$zipxSY(\{\{e_0, e_1, \dots\}, \dots, \{e'_0, e'_1, \dots\}\}) = \{[e_0, \dots, e'_0], [e_1, \dots, e'_1], \dots\}$
	unzipxSY	Constructs a process that converts a signal of vectors to a vector of signals	$unzipxSY(\{[e_0, \dots, e_{m-1}], \dots, [e'_0, \dots, e'_{m-1}]\}) = [\{e_0, e_1, \dots\}, \dots, \{e'_0, e'_1, \dots\}]$
Skeleton	*mapV*	A skeleton that applies a function f on each element of a vector	$mapV(f, [v_0, v_1, \dots, v_{m-1}]) = [f(v_0), f(v_1), \dots, f(v_{m-1})]$
	reduceV	A skeleton that reduces a vector to a single value via a two-input function	$reduceV(f, [v_0, v_1, \dots, v_{m-1}]) = f(v_0, f(v_1, f(\dots f(v_{m-2}, v_{m-1}))))$

example. The executable code of the application model is available at GitHub.[2] The functionality of the system is divided into the following processes:

- **graySY:** The input colorful image is received as a stream of pixels, and each pixel is represented by a triple of 3 integers (i.e., R, G, and B). `graySY` process groups the input stream into triples and converts each triple to a single grayscale pixel.
- **resizeSY:** This process averages each 2×2 subimage of the image down to one single pixel.
- **maxBrightnessSY:** This process identifies the brightest pixel in the grayscaled, resized image.
- **minBrightnessSY:** This process identifies the dimmest pixel in the grayscaled, resized image.
- **controlSY:** This process averages the differences between the brightest and dimmest pixels of three images, and if they are not in a certain range, enables the `correctionSY` process to adjust the brightness (i.e., automatic brightness control). This process is implemented by the `mooreSY` process constructor of ForSyDe.
- **correctionSY:** When enabled by `controlSY`, this process adjusts the brightness of the current image according to its maximum and minimum brightness.
- **sobelSY:** This process applies the Sobel operator and effectively performs edge detection. It outputs a signal that represents the brightness gradient magnitude for every pixel of the gray-scaled and resized input image.
- **asciiSY:** This process maps each pixel to an ASCII character depending on its value and outputs the ASCII art of the input image.

[2] https://github.com/forsyde/Transformation-Explorer.git

```
1  -- | Synchronous process network chaining a series of image processing
       algorithms
2  -- upon a stream of pixel values.
3
4  imageProcessingSY :: Signal (Image Int) -- ^ Input stream of pixel values
5                    -> Signal (Image Char) -- ^ Output stream of ASCII
       characters
6
7  imageProcessingSY inSig = processedImageSig
8    where
9      processedImageSig = asciiSY sobelSig
10     sobelSig = sobelSY correctedSig
11     correctedSig = correctionSY controlSig maxBrightnessSig
       minBrightnessSig resizedSig
12     controlSig = controlSY maxBrightnessSig minBrightnessSig
13     maxBrightnessSig = maxBrightnessSY resizedSig
14     minBrightnessSig = minBrightnessSY resizedSig
15     resizedSig = resizeSY  graySig
16     graySig = graySY  inSig
```

Listing 1. The net-list of the application model in Haskell/ForSyDe.

The model is a coarse-grained model of the system since the processes are hierarchies of processes and skeletons, and the functions are not basic logical operations.

Let's suppose that there is no pre-selected platform or component; then, based on the RAMP view of the system, the system model after the initial specification phase is $S_0 = (R_0, A_0, \emptyset, \emptyset)$, where A_0 is the initial application model shown in Figure 2, and the requirements are decomposed requirements.

4.2 Transformation Phase

The goal is to support transformation-based design by **design space exploration** (**DSE**) and estimation techniques, with the aim of selecting the transformations that can yield the most efficient implementations. The separation of concerns (Section 2.2) enables following the popular Kienhuis' Y-chart design approach [27] as a foundation for an efficient early-stage DSE in our methodology. The proposed methodology explores different application model structures defined by transformations, different platform models with various components, and different application-to-platform mappings. The exploration aims to results in the most promising sequence of transformations of the application model, the most suitable platform for the transformed application model based on that particular sequence, and the corresponding optimized mapping, while satisfying the requirements.

Figure 3 illustrates the transformation phase. This phase starts with the transformation space exploration. The application transformation space reflects different application structures. For the application model, rule-based transformations are used. Transformation rules can be formulated based on the semantics of the modeling frameworks explained in Section 2.1.

Example 4.2 (Transformation Rule). As an example of transformation rules, Figure 4 demonstrates the transformation rule "*MapMerge*" for process networks based on synchronous MoC, adapted from [40]. This transformation merges two processes. Any sub-process network in the application model with the format defined in the "Original Process Network" can be transformed to the format defined in the "Transformed Process Network" with no further implication, which means the semantics are preserved. This transformation potentially reduces vector traversals but requires that both processes be co-mapped to the same processor. Its inverse increases concurrency

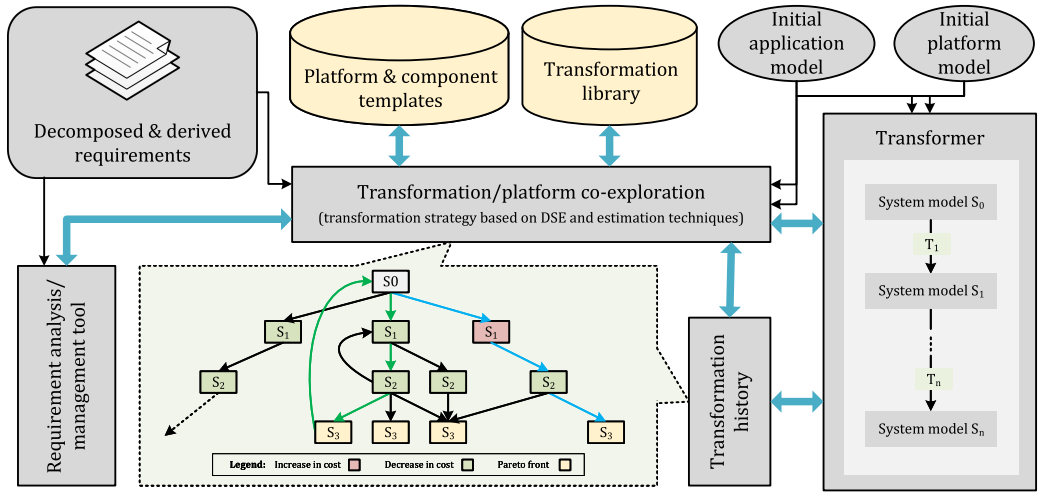


Fig. 3. The operational flow of the transformation phase. The dashed box illustrates the transformation tree and the possible phenomena in different sequences of transformations.

$$\text{Transformation Rule: } \textit{MapMerge}(PN)$$
$$\text{Original Process Network: } \overrightarrow{o} = PN(\overrightarrow{i}),\ PN(\overrightarrow{i}) = (mapSY(f) \circ mapSY(g))(\overrightarrow{i})$$
$$\text{Transformed Process Network: } \overrightarrow{o} = PN'(\overrightarrow{i}),\ PN'(\overrightarrow{i}) = (mapSY(f \circ g))(\overrightarrow{i})$$
$$\text{Implication: } \forall\, \overrightarrow{i}\,,\ PN'(\overrightarrow{i}) = PN(\overrightarrow{i})$$

Fig. 4. The transformation rule "*MapMerge*", adapted from [40].

by allowing the split processes to run on different cores in a pipeline manner. The choice between the transformation and its inverse depends on the computational workload.

As noted by Backus [2], modeling languages rooted in functional programming and utilizing "*combining forms*" for model construction (Section 2.1) are governed by associated algebraic laws. These laws enable *model transformations via function identities*. By leveraging such modeling languages, we can avoid reliance on low-level techniques to define transformation rules and instead utilize standard transformations. For example, the `Map-Split` transformation given in Example 4.2 is defined by adapting the *map distributivity* property of the HoF `map` to process constructor `MapSY` (Table 1) [39].

4.2.1 Pattern Matching. Starting with a high-level application model, the application transformation space is traversed by iterations of pattern matching and transformations. Using the transformation rules, the possible transformations are detected by matching the model to the patterns predefined in the transformation rules, for example, the "Original Process Network" in the transformation rule "*MapMerge*" shown in Figure 4 defines the pattern.

We propose an automatic pattern matching technique that explores an abstract graph representation of the system model and detects all possible transformations. ForSyDe IO meta-modeling framework proposed in [21] is used to capture the abstract graph representation. The framework is based on a formalism of system graphs and trait hierarchies and can be used for different modeling languages. *This framework is chosen because of its appropriateness for our transformational design process based on the RAMP view of the system. It captures the whole system model (RAMP) as a single common system graph across different design tools, which not only eliminates the need for*

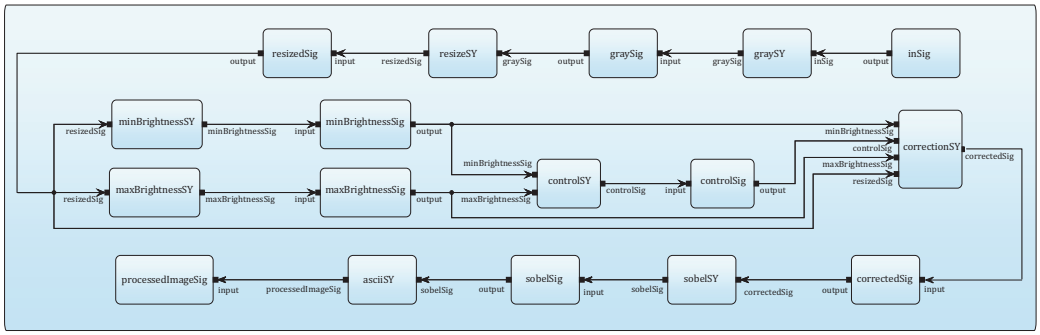


Fig. 5. The KIELER [36] visualization of the ForSyDe IO model of the image processing application (the traits and the hierarchies below the processes are abstracted for simplicity).

constant synchronization between models and facilitates seamless integration but also provides a common ground for design transformations (i.e., for transformations on different aspects of the design). ForSyDe IO framework also supports incremental changes, assuring that new modifications do not compromise the integrity of prior information. In addition, complex transformations can be constructed concretely by using the abstract system graph.

Since the specification model is based on the synchronous MoC and parallel data skeletons, a trait hierarchy for synchronous MoC and skeletons is created (Listing 3). The traits define the *required properties* and *ports* for vertices and edges of the system graph. For instance, in the trait hierarchy for synchronous MoC (`sy`), six traits are defined: `SYDataEdge` for edges connecting vertices that declare `SYElem`; `SYSignal` for signals, requiring one input and one output port connected to two vertices by `SYDataEdge` edges; `SYMap` for all processes except the ones constructed by the `delaySY` process constructor, requiring the input functions of the process; `SYDelay` for delay processes, requiring one input and one delayed output port connected to two signals by `SYDataEdge` edges.

Example 4.3 (Abstract Graph Representation). The structure of the image processing application model developed in ForSyDe/Haskell is captured using the ForSyDe IO framework. Figure 5 shows the visualization of *the abstract graph representation* of the image processing application developed using ForSyDe-IO [21] (the traits and the hierarchy below the processes are not shown for simplicity). The model is available in the implementation repositories.[3]

For pattern matching, first, the patterns are extracted from the transformation rules. The Haskell/ForSyDe code of the transformation rules for synchronous MoC (*transformation library*), including the original (patterns) and the transformed processes (transformed patterns), used in this article is given in Listing 2. All of the transformations in Listing 2 are semantic preserving. In the following, the rules are briefly explained:

— **Map-Split**: The reverse transformation rule of `MapMerge` (Figure 4), based on the identity `mapSY (f . g) = mapSY f . mapSY g` [41].
— **MapSY-ReduceV**: Used to convert a process that includes potential data parallelism (`reduceV`) into a parallel process network with the same function. It enables control over the degree of data parallelization.
— **MapSY-MapV**: Provides the same potential benefit as the rule `MapSY-ReduceV` does, for a process that includes skeleton `MapV`.

[3] https://github.com/forsyde/Transformation-Explorer.git

```
-- Transformation rules
-- Original Process = Transformed Process
-------------------------------------------------
-- Transformation Map-Split
mapSY (f . g) = mapSY f . mapSY g
-- Transformation MapSY-ReduceV
mapSY (reduceV f) = reduceV (zipWithSY f) . unzipxSY
-- Transformation MapSY-MapV
mapSY (mapV f) = zipxSY . mapV (mapSY f) . unzipxSY
-- Transformation ZipxSY-UnzipxSY
zipxSY . unzipxSY = id
-- Transformation Balanced-Tree
reduceV f vs = reduceTreeV f vs
reduceTreeV _ (a:>NullV) = a -- single element
reduceTreeV f vs =  -- tree stages recursion
  reduceTreeV f (mapV (reduceV f) (groupV 2 vs))
-- Transformation Serial-Clock-Domain
(delaySY s0 . mapSY (reduceV f) . zipxSY) vs =
(downDI (lengthV vs) . mooreSY u w (0, s0) . par2serxDI) vs
  where
    u (k,s) x | k == 0    = (1, x)
              | k == m-1  = (0, f x s)
              | otherwise = (k+1, f x s)
    w (k,s) = s
    m = lengthV vs
```

Listing 2. Transformation rules expressed as ForSyDe code.

— **ZipxSY-UnzipxSY**: Is based on the identity `zipxSY . unzipxSY = id`, and eliminates unnecessary compression and decompression tasks.
— **Balanced-Tree**: Transforms a combinational process to a balanced one. It can be used if the input function is combinational and its operator is associative [41].
— **Serial-Clock-Domain**: "Transforms a combinational process of a regular structure into a structure with two clock domains that uses an FSM process to schedule the operations into several clock cycles" [41]. In the transformed structure, the first clock domain operates at the original event rate, handling input values as they arrive, while the second clock domain schedules the serialized execution of the combinational function across multiple cycles. The `par2serxDI` (parallel to serial) domain interface reads and serializes the input values at event rate r, while the `downDI` interface down-samples the input signal, outputting the results at a reduced event rate to match the serialized computation. This necessitates having two distinct clock domains, one managing the high-rate input events and the other synchronizing the FSM-based process that executes the function over multiple cycles. Moreover, it is important to note that this transformation changes the scheduling of operations without altering the core functionality of the process. While the execution model shifts from a purely combinational approach to an FSM-based one, the combinational function itself remains intact, only distributed across multiple cycles. The focus of this transformation is on restructuring the execution timing.

These transformations are examples of the key categories of transformations that affect the structure of the process network. Numerous similar transformations can be formulated for a wide class of dataflow models as long as the causal behaviors implied from the chosen MoC are catered for, for example, when transposing between temporal and spatial domains. For example, Ungureanu [48] explains how the `zipx` processes are defined for different MoCs, which help to define `MapSY-MapV` and `MapSY-ReduceV` transformation rules (see Listing 2) for various MoCs.

ALGORITHM 1: Matching the patterns defined by the transformation rules of Listing 2 to the system graph:

Input: The system graph $G = (V, E)$, the trait hierarchy $\mathcal{T}$, transformation rules T
Output: Matched patterns $\mathcal{P}$
Begin:

```
1:  for v ∈ V do
2:      if v :: SYMap and dim(inputPorts(v)) = dim(outputPorts(v)) = 1 then
3:          if combFunctions(v) = [u] and u :: MapV then
4:              P ← P ∪ {MapSY-MapV(v, u)};
5:          else if combFunctions(v) = [u] and u :: ReduceV then
6:              P ← P ∪ {MapSY-ReduceV(v, u)};
7:              for d, c ∈ V with (d, c), (c, v) ∈ E and d :: SYDelay and c :: SYSignal do
8:                  P ← P ∪ {Serial-Clock-Domain(d, v, u)}
9:          else if combFunctions(v) = [u_1, ..., u_n] and n > 1 then
10:             P ← P ∪ {Map-Split(v, [u_1, ..., u_n])}
11:     else if v :: SYMap then
12:         if combFunctions(v) = [u] and u opens tuples then
13:             for z, c ∈ V with (z, c), (c, v) ∈ E and z :: SYMap and c :: SYSignal do
14:                 if combFunctions(z) = [y] and y makes tuples then
15:                     P ← P ∪ {ZipxSY-UnzipxSY(z, c, v)}
16:     else if v :: ReduceV then
17:         P ← P ∪ {Balanced-Tree(v)}
```

We show how automatic pattern matching is done in this example by taking advantage of the graph-theoretical nature of the chosen model. The pseudo-code of matching the patterns defined by the transformation rules of Listing 2 to the system graph is illustrated in Algorithm 1 (a Java implementation of Algorithm 1 is available at GitHub[4]).

Algorithm preliminaries: The algorithm is fed by the system graph $G = (V, E)$ and the trait hierarchy $\mathcal{T}$, and fetches the transformation rules T from the transformation library and gives the matched patterns $\mathcal{P}$. As explained in Section 2.1, the transformations modify the structure of the application model captured by means of the process constructors and skeleton generators. That is, the patterns define the format of the sub-process network that can be transformed in terms of these building blocks of the application model.

Therefore, to match the patterns, the vertices of the system graph are traversed, and their process constructors and skeleton generators are detected (Lines 1–16). In the algorithm, the operation $v :: t$ (Lines 2–16) is true if v conforms to a trait $t \in \mathcal{T}$, that is, declares to possess t and contains the required properties and ports.

Algorithm discussion: Targeting the transformations in Listing 2, Lines 2–4 match the pattern `MapSY-MapV`. If vertex v conforms to trait `SYMap`, and it is a single input single output vertex, the process constructor of the corresponding process to vertex v is `mapSY` (Line 2). For this combinational process (i.e., no internal state), the combinational function(s) is investigated (Lines 3, 5, and 9). If the process applies only one combinational function that conforms to `MapV` (Line 3), the pattern `MapSY-MapV` is matched (Line 4). Otherwise, if that only one combinational function conforms to `ReduceV` (Line 5), the pattern `MapSY-ReduceV` is matched (Line 6). The `mapSY (reduce V)` format is also part of the `Serial-Clock-Domain` pattern. To check if this pattern is also matched, the adjacent vertices d and c to vertex v that has the format `mapSY (reduce V)` are investigated. If vertex d that conforms to trait `SYDelay` is connected to vertex v through vertex c that conforms

[4] https://github.com/forsyde/Transformation-Explorer.git

ALGORITHM 2: Application of transformation `Map-Split` (Listing 2):

Input: The system graph $G = (V, E)$, the trait hierarchy $\mathcal{T}$, `Map-Split` matched pattern Map-Split$(v, [u_1, \ldots, u_n])$
Output: The transformed system graph $G' = (V', E')$
Begin:

1: $V' \leftarrow V \setminus \{v\}$
2: $E' \leftarrow E \setminus (\{(x, v) | x \in V\} \cup \{(v, x) | x \in V\})$
3: **for** $u_i \in \{u_1, \ldots, u_n\}$ **do**
4: $\quad V' \leftarrow V' \oplus \{\texttt{SYMap}(u_i)\}$
5: **for** $u_i \in \{u_1, \ldots, u_{n-1}\}, s_j \in V, (u_i, s_j), (s_j, u_{i+1}) \in E$ **do**
6: $\quad V' \leftarrow V' \oplus \{\texttt{SYSignal}(s_j)\}$
7: $\quad E' \leftarrow E' \oplus \{\texttt{SYDataEdge}(\texttt{SYMap}(u_i), \texttt{SYSignal}(s_j)), \texttt{SYDataEdge}(\texttt{SYSignal}(s_j), \texttt{SYMap}(u_{i+1}))\}$
8: $E' \leftarrow E' \cup (\{(x, \texttt{SYMap}(u_1)) | x \in V\} \cup \{(v, \texttt{SYMap}(u_n)) | x \in V\})$

to trait `SYSignal`, the pattern `Serial-Clock-Domain` is matched (Lines 7 and 8). However, if the corresponding process to vertex v applies more than one combinational function (Line 9), the pattern `Map-Split` is matched (Line 10). On the other hand, if vertex v conforms to trait `SYMap`, and applies only one combinational function that opens tuples (i.e., splitting an array input signal into an array of signals, e.g., `output1 = in[0]; output2 = in[1]; . . . outputN = in[N-1];`), the process constructor of the corresponding process to vertex v is `unzipxSY` (Line 11 and 12). Also, if vertex z conforms to trait `SYMap`, and applies only one combinational function that makes tuples (i.e., merging an array of signals into a single array signal, e.g., `output=(input1, input2, ..., inputN)`), the process constructor of the corresponding process to vertex z is `zipxSY` (Line 13 and 14). That is, if vertex v and vertex z are connected to each other by vertex c that conforms to `SYSignal`, pattern `ZipxSY-UnzipxSY` is matched (Line 15). Applying the data parallelization transformations like `MapSY-MapV` and `MapSY-ReduceV` may introduce many `ZipxSY-UnzipxSY` patterns to the model. Transformation `ZipxSY-UnzipxSY` eliminates unnecessary merges and splits. Finally, if vertex v conforms to trait `ReduceV`, pattern `Balanced-Tree` is matched.

After detecting the patterns, the model can be transformed by constructing a new sub-graph representing the transformed sub-process network, to be replaced with the original one in the transformed system graph. The new sub-graph is constructed based on the format defined in the applied transformation rule, for example, the "Transformed Process Network" in Figure 4.

4.2.2 Automatic Transformations. The general idea of new sub-graph construction is sketched for the transformation `Map-Split`. Algorithm 2 shows the pseudo-code of applying transformation `Map-Split` on the system graph.

Algorithm preliminaries: The algorithm is fed by the original system graph $G = (V, E)$, the trait hierarchy $\mathcal{T}$, and a `Map-Split` matched pattern Map-Split$(v, [u_1, \ldots, u_n])$, and gives the transformed system graph $G' = (V', E')$. First, the set of vertices V' is formed by vertices of V except vertex v to which pattern `Map-Split` is matched (Line 1). Similarly, the set of edges E' is formed by edges of E except for input edges and output edges of vertex v, that is, edges connecting vertex v to its preceding vertex and succeeding vertex, respectively (Line 2). *The original system graph is not directly manipulated since it is also an application instance and needs to be evaluated. Instead, a transformed graph is generated in which the sub-graph with the Map-Split format is replaced.*

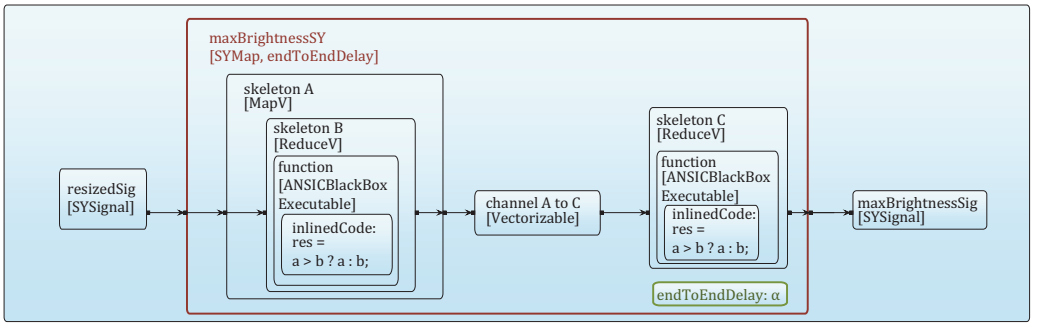
Algorithm discussion: For each combinational function of vertex v, that is, $u_i \in \{u_1, \ldots, u_n\}$, a new vertex based on u_i with the trait `SYMap` and a port connected to that combinational function is constructed and merged to the set of vertices V' (Lines 3 and 4). Due to the merging ($\oplus$) behavior defined in [21] , this construction and merge are equivalent to simply adding the required ports and

```
namespace sy {
    edge SYDataEdge {
        source SYElem
        target SYElem
    }
    vertex SYElem refines moc::MoCElem
    vertex SYProcess refines SYElem
    vertex SYSignal refines SYElem {
        port input is single in SYProcess with SYDataEdge
        port output is single out SYProcess with SYDataEdge
    }
    vertex SYMap refines SYProcess {
        prop inputPorts is array<str>
        prop outputPorts is array<str>
        port combFunctions is multiple out impl::Executable with moc::
AbstractionEdge
    }
    vertex SYDelay refines SYProcess {
        prop numDelays is int
        port normal is single in SYSignal with SYDataEdge
        port delayed is single out SYSignal with SYDataEdge
    }
}
namespace parallel {
vertex ParallelSkeleton refines impl::Executable
edge ParallelContainerEdge
edge ParallelComputationEdge
vertex Vectorizable {
    prop dimensions is array<int> = []
    }
vertex Map refines ParallelSkeleton {
    prop inputPorts is array<str>
    prop outputPorts is array<str>
    port kernel is single output impl::Executable with
ParallelContainerEdge
    }
vertex Reduce refines ParallelSkeleton {
    prop inputArrayPortName is str
    prop outputScalarPortName is str
    port kernel is single output impl::Executable with
ParallelContainerEdge
    }
}
```

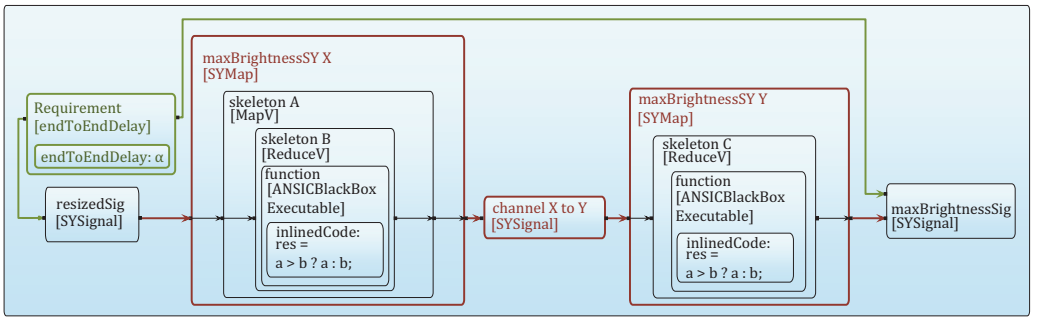
Listing 3. Excerpt of the case study trait hierarchy description created using the ForSyDe IO meta-modeling framework [21].

properties for SYMap on u_i. For all constructed vertices for the adjacent combinational functions u_i and u_{i+1}, a connecting vertex with the trait SYSignal is constructed and merged to the set of vertices V' (Lines 5 and 6). Connecting edges of the constructed vertices in Lines 4 and 6 are generated with the trait SYDataEdge, that is, since they connect vertices that declare SYElem, and are merged to the set of edges E' (Line 7). Finally, the input edges to the original vertex v and its output edges are connected to the first constructed vertex with the combinational function u_1 and the last constructed vertex with the combinational function u_n, respectively (Line 8).

Figure 6 illustrates and visualizes the application of the above algorithm on maxBrightnessSY process that has the format defined in pattern Map-Split. *This figure also exemplifies another advantage gained because of the carefully selected underlying graph representation; providing a common*



(a) The original process `maxBrightnessSY`.



(b) The transformed process `maxBrightnessSY`.

Fig. 6. Applying the transformation `Map-Split` on process `maxBrightnessSY` and construction of the transformed sub-process network. The transformed graph is annotated by requirement $endToEndDelay = \alpha$ by adding a new vertex to the graph. The figures are automatically generated using KIELER by the adaptor of ForSyDe IO. The colored boxes highlight the changes in the correlated parts after the transformation. Traits, ports, and properties are simplified for clarity.

infrastructure for transformations on different views of the design. ForSyDe IO meta-modeling framework allows annotating the application graph with resource requirements and constraints. These are captured either as properties of individual vertices or, for requirements involving relationships between multiple vertices, as additional vertices representing those requirements. As can be seen, the vertex `maxBrightnessSY` is annotated with the timing requirement $endToEndDelay = \alpha$, which expresses the end-to-end delay requirement. In order to keep this requirement after the transformation, a new vertex with the trait `endToEndDelay`, the property $endToEndDelay = \alpha$, and ports `initialPort` and `endPort` is constructed and added to the transformed system graph. The `initialPort` is connected to the preceding vertex of the original vertex and the `endPort` to its succeeding vertex in the transformed graph. A more advanced transformation can annotate the vertices of the replaced sub-graph with the decomposed end-to-end delay requirement. Depending on the requirement, similar techniques can be adopted to annotate the transformed system graph with any other type of requirement after transformations.

The iterations of pattern matching and transformations result in different sequences of transformations, each of which generates an application instance (i.e., the nodes of the *transformation tree* in Figure 3) reflecting various application structures. That is, the transformations define the application space. To efficiently define the application space, the application of transformations needs to be controlled. For example, because of the inverse relationship between some transformation rules (e.g., `Map-Merge` and `Map-Split`), the sequences of transformations leading

Table 2. Pattern Recognition and Transformation Steps of Figure 7

Transformation step	Detected patterns	Selected transformation	Applied transformation
❷	Balanced-Tree, Map-Split	Map-Split	❷ $\xrightarrow{T_m}$ ❸
❸	Balanced-Tree, MapSY-MapV, MapSY-ReduceV	MapSY-ReduceV	❸ $\xrightarrow{T_{m+1}}$ ❹
❹	Balanced-Tree, MapSY-MapV	MapSY-MapV	❹ $\xrightarrow{T_{m+2}}$ ❺
❺	Balanced-Tree, MapSY-ReduceV, ZipxSY-UnzipxSY	ZipxSY-UnzipxSY	❺ $\xrightarrow{T_{m+3}}$ ❻
❻	Balanced-Tree, MapSY-ReduceV	MapSY-ReduceV	❻ $\xrightarrow{T_{m+4}}$ ❼
❼	Balanced-Tree	Balanced-Tree	❼ $\xrightarrow{T_{m+5}}$ ❽

```
1  -- The Original Process
2  maxBrightness = mapSY (reduceV f . mapV (reduceV f))
3  -- Transformation Map-Split
4  maxBrightness = mapSY (reduceV f) . mapSY (mapV (reduceV f))
5  -- Transformation MapSY-ReduceV
6  maxBrightness = reduceV (zipWithSY f) . unzipxSY . mapSY (mapV (reduceV f))
7  -- Transformation MapSY-mapV
8  maxBrightness = reduceV (zipWithSY f) . unzipxSY . zipxSY . mapV (mapSY (
       reduceV f)) . unzipxSY
9  -- Transformation ZipxSY-UnzipxSY
10 maxBrightness = reduceV (zipWithSY f) . mapV (mapSY (reduceV f)) . unzipxSY
11 -- Transformation MapSY-ReduceV
12 maxBrightness = reduceV (zipWithSY f) . mapV (reduceV (zipWithSY f) .
       unzipxSY) . unzipxSY
```

Listing 4. The transformation of process maxBrightnessSY.

to the same model should be avoided. The green branch in the transformation tree of Figure 3 shows this phenomenon. This figure also demonstrates other phenomena that may happen in design transformations, for example, infinite branches or different branches resulting in the same model.

Example 4.4 (Application Instances). Figure 7 visualizes the pattern matching of transformation rules of Listing 2 to the process maxBrightnessSY and the application of the transformation sequence $T_m - T_{m+4}$. Patterns are shown by means of transparent colored boxes. Step ❶ shows the original process. Pattern recognition and transformation start from Step ❷. Table 2 summarizes detected patterns and applied transformations in each step of Figure 7. The result of each step is an application instance, and changing the selected transformations in Table 2 leads to different instances. Listing 4 shows the associated transformation of the source code. The source code of the transformed application model is available at GitHub.[5]

4.2.3 Efficient Sequence of Transformations. The sequence of transformations that can yield the most efficient implementation is selected by DSE support. That is because the applied sequence of transformations highly affects the final implementation. After capturing the application space, a DSE technique can be used to jointly explore the application space (the transformation space), the platform space, and the mapping space, with respect to the requirements. The DSE process finds the RAMPs that make the best trade-off between different performance metrics. The DSE tool can employ a variety of approaches, including both heuristic (Section 5) or exact methods, analytical or non-analytical, and can range from having no optimization objectives to handling multiple objectives.

[5] https://github.com/forsyde/Transformation-Explorer.git

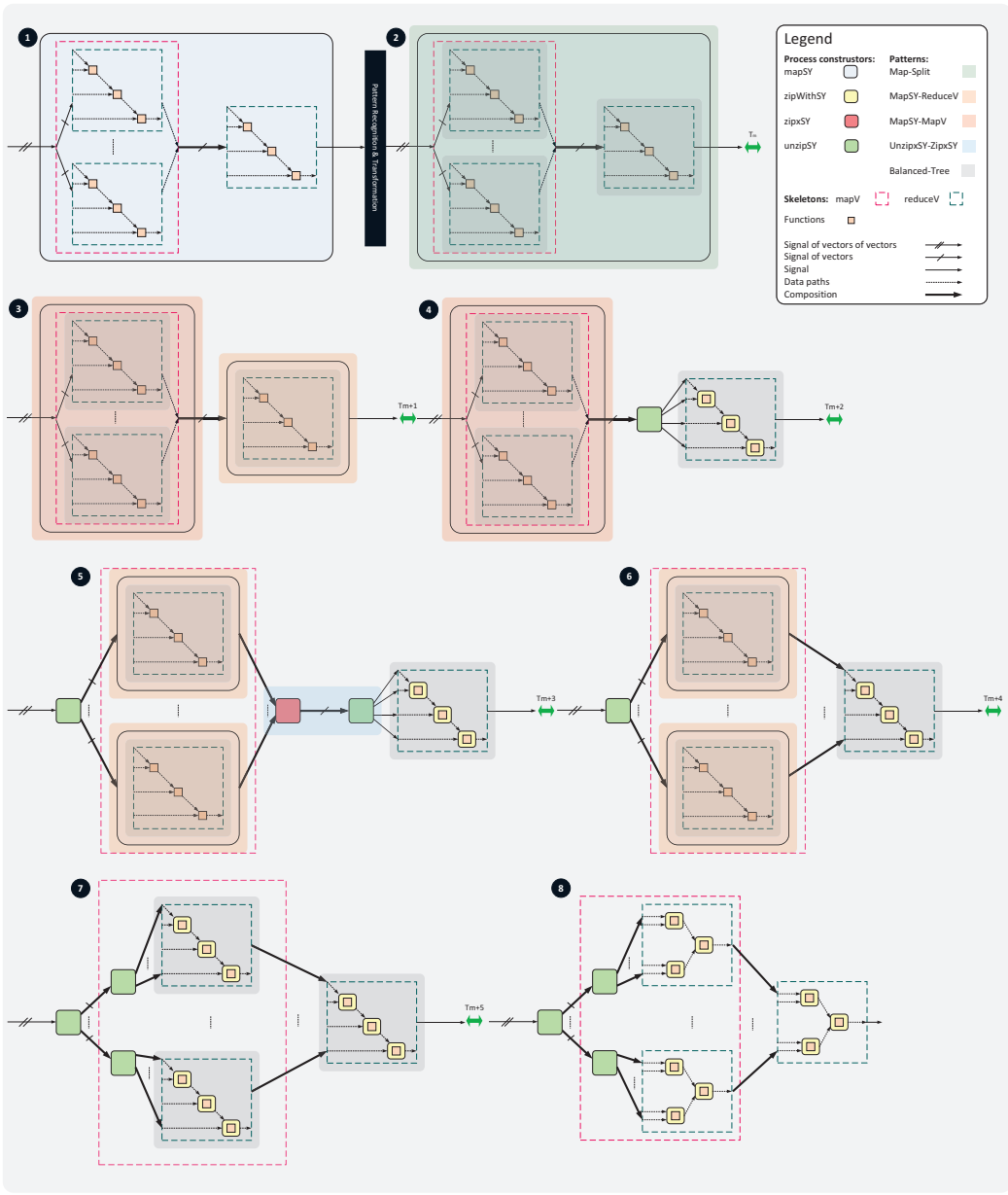


Fig. 7. Pattern detection and transformation of `maxBrightnessSY` function.

Example 4.5 (Sequences of Transformations). Table 3 shows the application of three different sequences of transformations on the part of the transformed process `maxBrightnessSY`, that is, `reduceV (mapSY f)`. Figure 8 illustrates the system model resulting from each of the sequences. This example illustrates transformations on all views of the system model. For each sequence of transformations, Table 3 shows how the original system model, represented in the RAMP format, is transformed by a step-wise application of transformations. In the first sequence, the transformation rule `Balanced-Tree` is used, and hardware implementation is preferred. However, in the

Table 3. Three Different Sequences of Transformations of Process `maxBrightnessSY`

Original system model: $S_n = (R_i, A_j, M_k, P_l), A_j : reduceV(zipWithSYf), R_i : t_{prop} \leq 60$		
Sequence 1	**Sequence 2**	**Sequence 3**
-	-	$S_n = (R_i, A_j, M_k, P_l)$ $A_j \xrightarrow{\texttt{MapSY-ReduceV}} A_{j+1}$
$S_n = (R_i, A_j, M_k, P_l)$ $A_j \xrightarrow{\texttt{Balanced-Tree}} A_{j+1}$	-	$S_n = (R_i, A_{j+1}, M_k, P_l)$ $A_{j+1} \xrightarrow{\texttt{Serial-Clock-Domain}} A_{j+2}$
$S_n = (R_i, A_{j+1}, M_k, P_l)$ $P_l \xrightarrow{HW} P_{l+1}$	$S_n = (R_i, A_j, M_k, P_l)$ $P_l \xrightarrow{HW} P_{l+1}$	$S_n = (R_i, A_{j+2}, M_k, P_l)$ $P_l \xrightarrow{SW} P_{l+1}$
$S_n = (R_i, A_{j+1}, M_k, P_{l+1})$ $M_k \xrightarrow{A_{j+1}, P_{l+1}} M_{k+1}$	$S_n = (R_i, A_j, M_k, P_{l+1})$ $M_k \xrightarrow{A_j, P_{l+1}} M_{k+1}$	$S_n = (R_i, A_{j+2}, M_k, P_{l+1})$ $M_k \xrightarrow{A_{j+2}, P_{l+1}} M_{k+1}$
$S_n = (R_i, A_{j+1}, M_{k+1}, P_{l+1})$ $R_i \xrightarrow{decompose} R_{i+1} : 2 * t_t \leq 60$	$S_n = (R_i, A_j, M_{k+1}, P_{l+1})$ $R_i \xrightarrow{decompose} R_{i+1} : 3 * t_t \leq 60$	$S_n = (R_i, A_{j+2}, M_{k+1}, P_{l+1})$ $R_i \xrightarrow{decompose} R_{i+1} : t_{C-Code} \leq 60$
$S_n = (R_{i+1}, A_{j+1}, M_{k+1}, P_{l+1})$	$S_n = (R_{i+1}, A_j, M_{k+1}, P_{l+1})$	$S_n = (R_{i+1}, A_{j+2}, M_{k+1}, P_{l+1})$

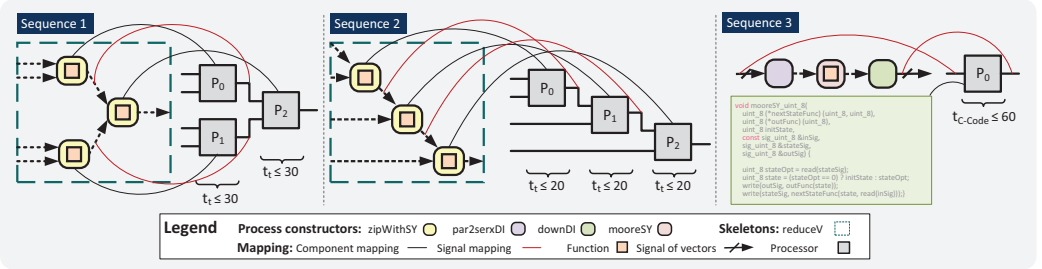


Fig. 8. System models resulted from different sequences of transformations shown in Table 3.

second sequence, a hardware implementation of the original application model (i.e., no transformation or identity transformation) is selected. Finally, in the third sequence, after transformation `Serial-Clock-Domain` (Listing 2), the application model includes the process `mooreSY (max) (id) 0` constructed by process constructor `mooreSY`, which is interpreted as an FSM [41]. The software implementation (C-code) of the application model is mapped to a single core. Figure 8 also shows the requirement decomposition, where the top-level requirement $R_i : t_{prop} \leq 60$ is further decomposed to be assigned to the individual sub-systems. A DSE technique can automatically generate these design points based on our proposed transformational design methodology and the pattern matching and transformation framework. The DSE also finds the Pareto front RAMPs for a set of optimization objectives.

5 Experiments

This section demonstrates the proposed design flow using two use cases: first, a subset of the image processing application (Example 4.1) is implemented on a tile-based platform, and second, the same platform is used to implement a cloud detection application.

The platform is a single bus platform and consists of seven identical tiles designed for general-purpose computing tasks alongside a dedicated accelerator tile (Figure 11). This accelerator is specifically engineered to handle the `sobelSY` process of the image processing application and the `multiplyBandsSY` process (Section 5.2) of the cloud detection application. In this TDMA-based communication system, the time axis is discretely divided into time slots. Each tile can reserve a

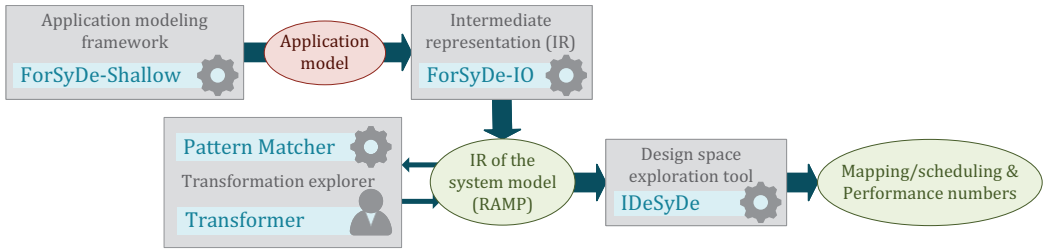


Fig. 9. Tool chain overview: components and workflow.

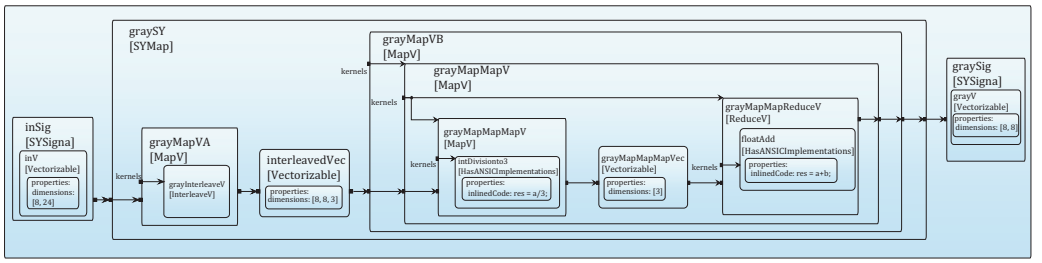


Fig. 10. The KIELER visualization of the ForSyDe IO model of part of the image processing application. A comprehensive view of the hierarchical structure below graySY process, encompassing all levels down to the basic functions.

dedicated time slot, ensuring uninterrupted access to the bus for data transmission or reception. The reservation of time slots to tiles is done in the DSE phase. The execution of computational tasks on tiles does not have to wait for communication tasks, as they can be independently done through the interconnect of the bus. This design choice enables a *predictable* platform suitable for safety-critical real-time systems. It also fosters a more flexible platform that is easier to scale [10]. Blocking communication (i.e., blocking send- and receive-primitives) between tiles is offered by the platform. Furthermore, we assume that all tiles, including the accelerator tile, are capable of executing a repetitive, static, order-based schedule. Tiles are also equipped with local memories.

The primary objective of the experiments is to assess the effect of the applied transformations. Figure 9 shows the tool chain used for the case study. The application is modeled as a synchronous process network using the ForSyDe-Shallow[6] modeling framework. The abstract graph representation of the system model consisting of the intermediate representations of the application model (Figures 10 or 14) and platform model (Figure 11), as well as the resource requirements, are captured using the ForSyDe IO meta-modeling framework. The developed pattern matching tool matches the patterns defined in the transformation rules to the system graph and automatically finds all possible application transformations. The transformations are applied manually. Automation of the application of transformations is a natural direction for future work. The transformed system graphs are fed to the DSE tool IDeSyDe [23]. IDeSyDe finds the application to platform mappings and scheduling, add them to the system graph, and calculates corresponding performance numbers for all transformed graphs.

IDeSyDe is an extensible meta-DSE framework based on the **design space identification** (**DSI**) approach proposed in [22, 23]. Due to its DSI foundation, IDeSyDe enables the reuse of different DSE models and DSE algorithms. More importantly, IDeSyDe contains pre-existing modules with

[6]https://github.com/forsyde/forsyde-shallow

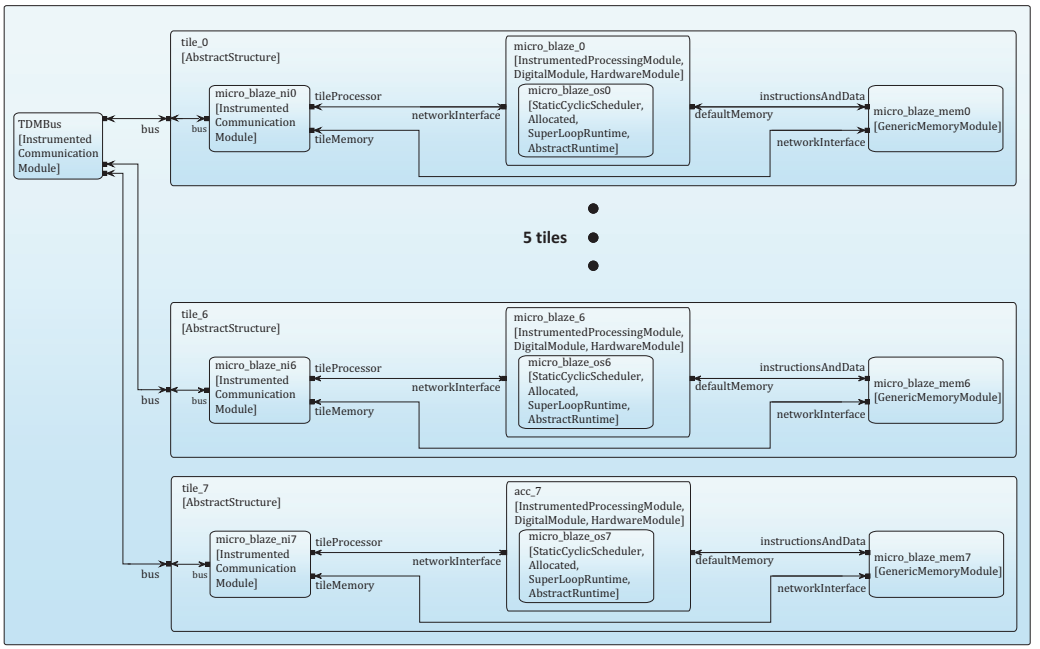


Fig. 11. The KIELER visualization of the ForSyDe IO model of the case study platform. The figure represents a simplified view of the tile-based platform featuring 2 out of the 7 homogeneous tiles, along with a dedicated accelerator.

DSE models and algorithms for DSE problems it was used to solve. One of these DSE scenarios is the mapping/scheduling of ForSyDe IO synchronous process networks to tile-based platforms, which we use in this case study. The Jenetics GA framework[7] is used to create a search algorithm for this DSE scenario. The optimization goals of the algorithm are minimizing the inverse throughput and the number of tiles used. Optimization of the number of tiles used is an example of platform transformation. The tiles used are estimated in a course-grain fashion by counting the number of PEs used, while the inverse throughput is estimated through SDF methods [23, 28] applied to the application graph, once it is mapped and order-based scheduled in the target platform.

Our current focus is not on optimizing the transformation selection strategy; refining it will be part of our future work. Currently, our primary objective is the collection of relevant data and insights. Given our interest in transformations that extract potential parallelization, three transformations `Map-Split`, `MapSY-MapV`, and `MapSY-ReduceV` from Listing 2 are considered. These transformation rules exhibit no overlap; it never occurs that more than one pattern is matched to a process. Therefore, the transformation tree converges, and the exhaustive exploration of all branches is unnecessary. Starting from the initial application graph, each iteration involves the identification of potential transformations using the pattern matching tool. A random selection is made, and the chosen transformation is applied. This iterative process is repeated on the resulting transformed application graph until *no further transformations are feasible*.

As explained in Section 4.2.1 transformations `MapSY-MapV` and `MapSY-ReduceV`, "transform a process containing application of a vector iterator into a vector of processes" [38]. We will refer to the vectors of processes resulting from these transformations as *parallel structures*, and we define

[7] https://jenetics.io/

the size of a parallel structure as the number of processes it contains (i.e., a vector of n processes is a parallel structure of size n, denoted as $|parallel\ structure| = n$). When a pattern is matched to any process within a parallel structure, it inherently matches all other processes within that structure. However, applying the corresponding transformation rule of that pattern to the first process exclusively, or alternatively to the second process, yields identical transformed graphs. To ensure uniqueness, we have applied symmetry breaking to exclude such equal transformed graphs. Nevertheless, applying the transformation rule to only one process within a parallel structure or to two processes within the same structure generates distinct transformed graphs, each characterized by different levels of parallelization, all of which are considered in our experiments.

5.1 Image Processing Application

We explore the entire transformation space of the `graySY` process of the image processing application (Figure 10). The outcomes of our experiments are illustrated in Figure 2. The iterative detection and application of the three transformations (i.e., `Map-Split`, `MapSY-MapV`, and `MapSY-ReduceV`) on the `graySY` process yield a total of 194 transformed graphs. Application instance number 0 represents the initial application graph. Subsequent instances, such as application instance number 1, result from applying one transformation rule on the preceding instance (in this case, on application instance number 0). This sequential progression continues until reaching Application instance number 194, signifying 194 successive transformations.

The application instances are subsequently analysed by IDeSyDe, which identifies an efficient mapping and schedules for the given application graph and returns performance numbers. As an example of the mapping model, Figure 12 shows the KIELER visualization of the ForSyDe IO model of the mapping of the first transformed graph. The outcomes of the DSE phase are illustrated in Figure 13. In Figure 13(a), the time-out for refining the current best exploration solution is configured for 1 minute. This implies that if no Pareto-dominant solution is identified within this time frame, the exploration process terminates. The experiments are repeated for an improvement time-out of 5 minutes (Figure 13(b)). Smaller values for inverse throughput are better in Figure 13. Notably, some application instances pose challenges for IDeSyDe to find a solution within the specified time-out, and these instances, therefore, are not included in the presented plot. Also, the outcomes for the initial application model ($|PEs| = 1, invThroughput = 363520$) and the first transformed graph ($Pareto - point\ 1 : |PEs| = 1, invThroughput = 363520$ and $Pareto - point\ 2 : |PEs| = 2, invThroughput = 360448$) are omitted from the plot due to their significantly larger inverse throughput values from the subsequent results, and for a clearer and more coherent visualization. Based on the abstraction within IDeSyDe and applied transformation rules, we expect that transformations should either maintain or enhance performance metrics. Our hypothesis centers on the potential impact of these transformations on communication time. If the extracted parallelization by transformations adversely affects system throughput (possibly due to increased communication time), there is the flexibility to map all newly generated processes resulting from the transformation to the same core. Hence, the deterioration in performance metrics displayed on the plot is likely because of the imposed time-out for improvement. The limitations in exploration time have prevented a comprehensive assessment and improvement of the solutions.

For lower application instance numbers, the solutions found are, on average, less optimal, while the time required for new Pareto improvements is larger. This is likely due to the changes in symmetry as transformations are applied to the application graph. The asymmetry makes it more challenging for the DSE tool to converge to near-optimal solutions quickly and improve the current solution. The first two transformations in the sequence, leading to the application instances depicted in Figure 13, involve applying the `Map-Split` transformation rule on the `graySY` process, followed by applying `MapSy-MapV` transformation rule on one of the processes generated by the

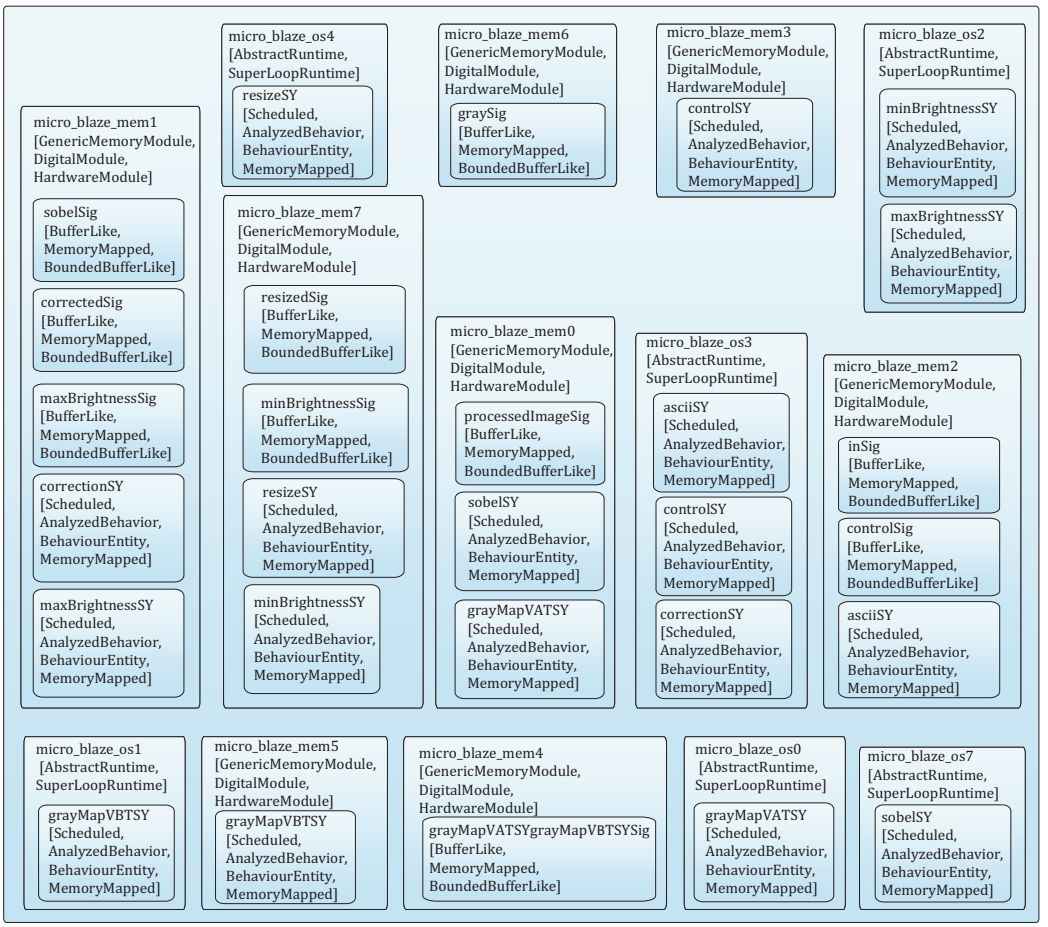
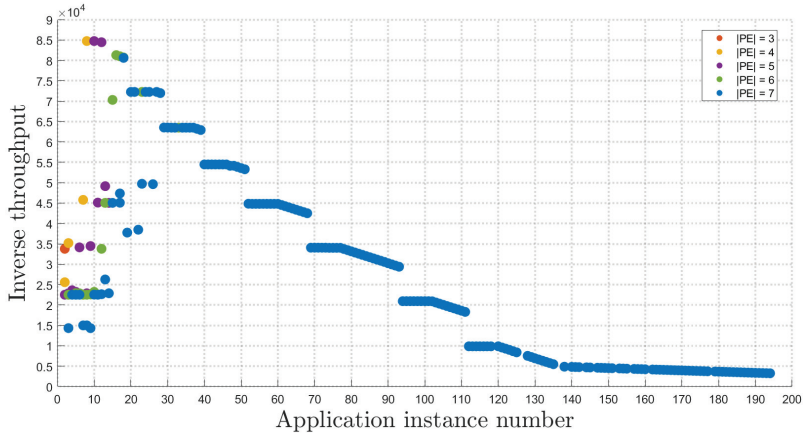


Fig. 12. The KIELER visualization of the ForSyDe IO model of the mapping of the application instance 1 on the platform. This application instance is the result of applying transformation `Map-Split` on the `graySY` process of the image processing application model. To enhance clarity, non-essential visual elements unrelated to the mapping are excluded.

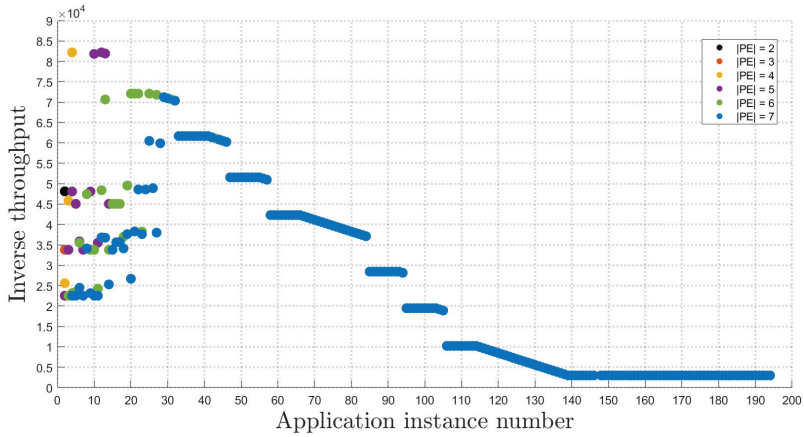
first transformation, creating a parallel structure of size 8. After the second transformation, one of the processes within the generated parallel structure undergoes iterative transformation until no further transformations are feasible for that process. The transformation sequence then continues with the other processes in that parallel structure in the same manner. Hence, the observed initial deterioration in performance metrics can be due to the increased asymmetry introduced during the first transformations. Subsequently, at later iterations and higher granularity, graphs eventually regain symmetry. Defining a strategy that transforms either none or all processes within a parallel structure in a single step can effectively reduce the transformation space, thereby accelerating the DSE process.

5.2 Cloud Detection Application

The cloud detection application is based on the algorithm described in [42]. It processes satellite imagery using three RGB bands (red, green, and blue) as input signals to detect cloud-covered



(a) $improvment - time - out = 1\ minute$



(b) $improvment - time - out = 5\ minutes$

Fig. 13. Resulting in inverse throughput and the number of used PEs ($|PE|$) for the mapping/scheduling of different application instances (transformed graphs) of the image processing application on the platform. Lower inverse throughput is better. Each point plotted is a Pareto-solution returned by the DSE tool IDeSyDe.

areas. Figure 14 shows the visualization of the ForSyDe IO model for the synchronous process network of the application. The ForSyDe IO model of the application is available at GitHub.[8] The functionality of the application is divided into the following processes:

— **threshold1SY, threshold2SY, and threshold3SY:** Each image band (red, green, and blue) undergoes a threshold operation with a threshold value τ_1. This step helps highlight areas with brightness levels that might indicate clouds, producing binary matrices for each band.
— **multiplyBandsSY:** The binary matrices are multiplied element-wise, combining the information from the three bands. This operation enhances the detection of potential

[8] https://github.com/forsyde/Transformation-Explorer.git

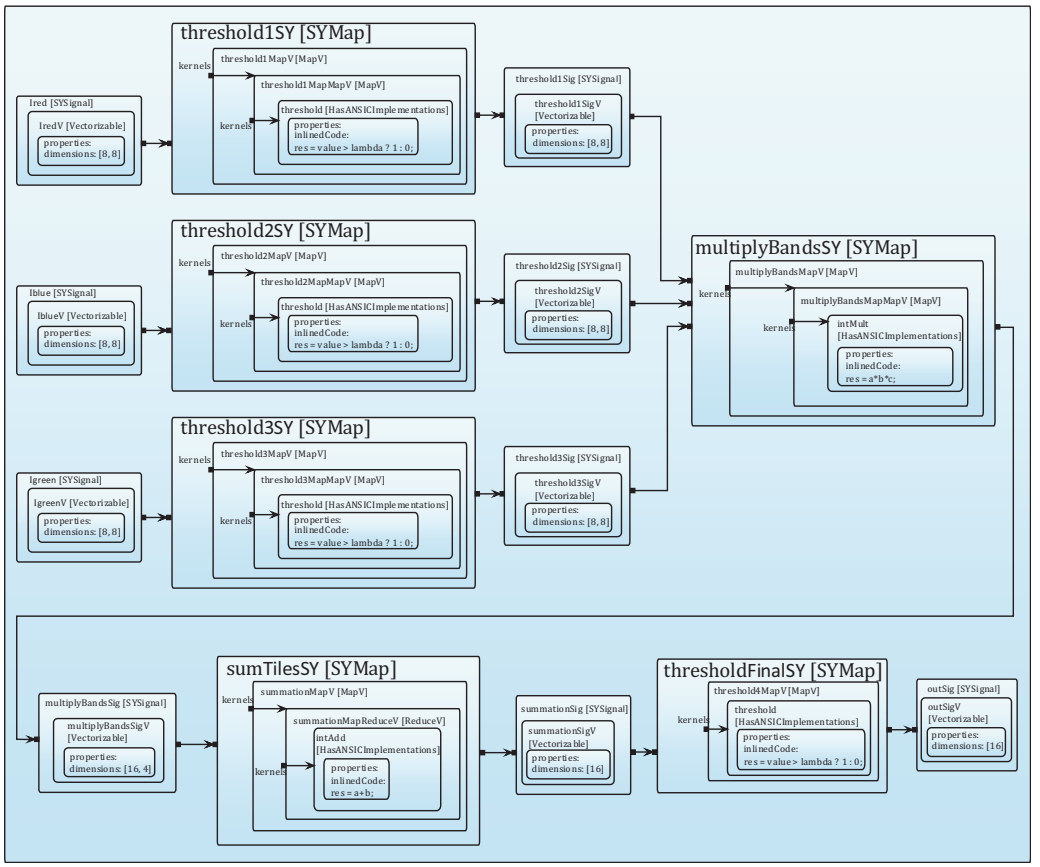


Fig. 14. The KIELER visualization of the ForSyDe IO model of the cloud detection application. Traits, ports, and properties are simplified for clarity.

cloud-covered areas, as it identifies pixels where all three bands simultaneously indicate high brightness.

— **sumTilesSY:** The images are divided into tiles ($\delta \times \delta$) and the resulting values from the multiplication step are summed within each tile. The total brightness per tile is calculated and normalized by dividing the sum by the area of the tile (δ^2), producing a single brightness value for each tile.

— **thresholdFinalSY:** A second threshold operation with value τ_2 is applied to the calculated brightness ratio for each tile. Tiles are classified as either "cloudy" or "no cloud" on the basis of this threshold, yielding a final binary flag for each tile.

Our pattern matching technique and transformations effectively identify and leverage the potential for parallelization within the cloud detection algorithm. This parallelization facilitates faster processing of large satellite datasets, which often consist of extensive images that require the analysis of multiple bands for each pixel. The initial model of the application already incorporates a tile-based structure, making it inherently suitable for parallel execution. The transformations enable independent execution of tiles across different PEs in parallel. Furthermore, the computationally intensive task of multiplying image bands is efficiently mapped and executed on the accelerator tile.

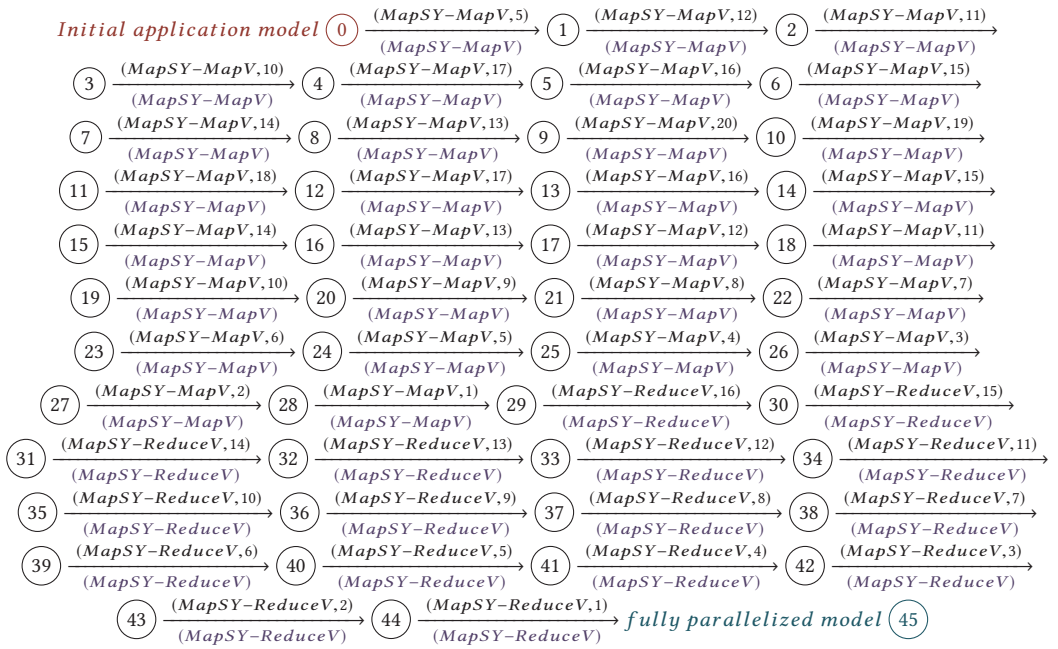
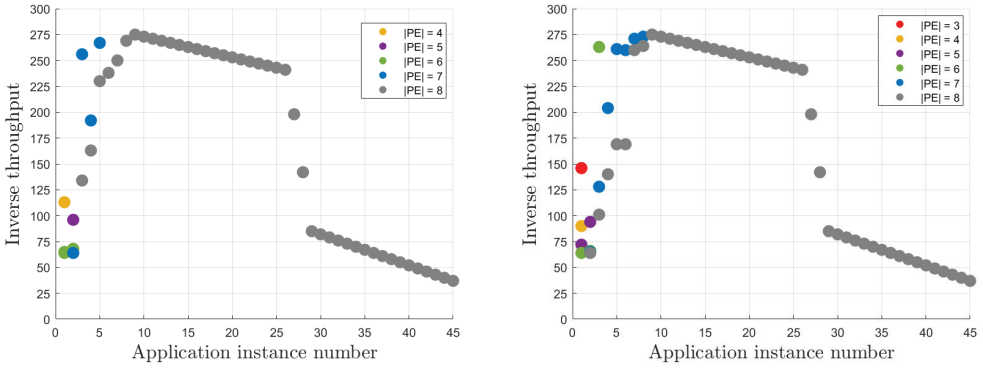


Fig. 15. Transformation sequence applied on cloud detection application.

Figure 15 illustrates the sequence of transformations applied to various processes within the cloud detection application. Arrows in the figure represent the application of transformation rules. Above the arrows, the patterns identified are represented as (x, n), where x refers to the pattern and n denotes the number of times it is matched. Below the arrows, the specific transformations that are selected and applied are shown. The parallelization process begins by identifying five `MapSY-MapV` patterns (corresponding to processes `treshold1SY`, `treshold2SY`, `treshold3SY`, `sumTilesSY`, and `tresholdFinalSY`, all of which follow the `MapSY-MapV` format). One of these is then transformed, producing the graph labeled (1). In this updated graph, 12 occurrences of the *MapSY-MapV* pattern are detected, and another transformation is applied. This iterative process continues until no more transformations can be performed, resulting in a total of 45 transformed graphs.

The transformed graphs are subsequently analyzed by IDeSyDe. The outcomes of the DSE phase are illustrated in Figure 16. The experiments are done for an improvement time-out of 1 minute and 5 minutes. As with the results for the image processing application (Figure 13), the applied transformations are expected to maintain or improve performance metrics. Any decline in performance, particularly due to increased communication time, could be mitigated by mapping the newly generated processes to the same core. The observed reduction in performance is likely due to the time-out imposed on the DSE process, limiting further exploration and optimization.

Furthermore, similar to the results for the image processing application, the applied transformations introduce asymmetry, making it more challenging for the DSE tool to converge on near-optimal solutions or improve existing ones quickly. However, as iterations progress and graph granularity increases, symmetry is gradually restored, leading to a more efficient exploration by the DSE tool. To test this hypothesis, we applied transformations that systematically increase asymmetry up to a certain point. Specifically, the first three processes `threshold1SY`, `threshold2SY`, and `threshold3SY` were each transformed into a parallel structure of size 8. Next, we transformed the first process in each of these parallel structures one after another, followed by transforming



(a) $improvment - time - out = 1\ minute$ (b) $improvment - time - out = 5\ minutes$

Fig. 16. Resulting in inverse throughput and the number of used PEs ($|PE|$) for the mapping/scheduling of different application instances (transformed graphs) of the cloud detection application on the platform. Lower inverse throughput is better. Each point plotted is a Pareto-solution returned by the DSE tool IDeSyDe.

the second process of each parallel structure in the subsequent transformations, rather than transforming all processes within the same parallel structure consecutively. This continued until transformations 26, 27, and 28, which produced transformed graphs (27), (28), and (29), respectively, (Figure 15). The three notable improvements in inverse throughput for these transformed graphs in Figure 16 correspond to the final transformations of processes `threshold1SY`, `threshold2SY`, and `threshold3SY` (i.e., the last processes in each parallel structure), where the graph regains substantial symmetry.

The results demonstrate significant performance improvements in the final transformed graphs compared to the initial graphs for both the image processing and cloud detection applications. Additionally, the transformations not only broaden the design space but also guide the DSE tool toward more optimal regions within that space. Despite the larger size of the final transformed graphs, the DSE tool still identifies solutions with superior performance metrics within the same timeout period. These case studies underscore the methodology's applicability, though its full potential will be realized with automated transformations, enabling more comprehensive experimentation and validation.

6 Related Work

Our work mainly builds on two research areas: rule-based program transformation techniques and **electronic system-level** (**ESL**) design methodologies.

A considerable body of research on program transformation exists in the literature [34, 49, 51]. However, such techniques do not consider the interplay of tight constraints and platform heterogeneity inherent in embedded systems and; therefore, their application on system design is not straightforward. More closely related to our work, there are a number of works on transformation techniques for functional programs (i.e., since process constructors and skeletons are defined as HOFs (Section 2.1)) [35]. Nevertheless, because of the lack of familiarity with program transformation techniques among system designers and the popularity of model-based design, model transformations are often preferred. Reekie [38] provides semantics for formal description of dataflow process network models, which are widely used in system design. The functional programming language Haskell is used for coding such models and, therefore, provides the ground for

applying the functional programming transformation techniques to process network models. Sander et al. [41] extend Reekie's work by adding the support for non-semantic preserving transformations, which are essential in a high-level design approach to refine the models. However, these works lack a mechanized and potentially automatable technique for the detection, selection, and application of transformations, that also considers the embedded system design challenges.

In addition to the above-mentioned works, the previous literature on *development by refinement*, firstly advocated by [13] and later fully formalized by [1], are also orthogonal to the current article. More recently, Broy et al. [7] and Nuzzo et al. [33] have proposed design methods based on the development by refinement approach for interactive and **cyber-physical systems** (**CPS**), respectively. Nuzzo et al. [33] combine **platform-based design** (**PBD**) [15] with contracts. PBD-based design methodologies focus on abstraction and refinement, following a "meet-in-the-middle" strategy. In the bottom-up phase, a component library is constructed by abstracting potential implementations, while in the top-down phase, a high-level specification is gradually refined and mapped to components from the library. These methodologies mainly focus on the refinement process that makes a system more specific or detailed by adding constraints, allowing the level of abstraction to be moved down [4]. In contrast, transformations often work within the same abstraction level, focusing on enhancing the quality of the design. Typically, the system's behavior is preserved, but specific aspects of the design are improved. For example, optimization transformations aim to improve performance or resource usage [11, 45]. Transformations typically involve modifications to the computational structure, optimizing the number and types of computational modules, their connections, and the order of operations [9].

Moreover, Nuzzo et al. [33] employ contracts to define and abstract the computation and communication components of a CPS. Following a PBD-based design approach, these components are typically viewed as predetermined or, at the very least, significantly constrained. Our work diverges from this in a key way. Our transformational design approach is platform-independent, allowing for optimizations that are not tied to any specific hardware or software platform. For instance, the transformations `MapSY-MapV` and `MapSY-ReduceV` (see Listing 2) enhance concurrency levels without being tied to a specific platform. This flexibility allows for a broader and more dynamic DSE, where various concurrency levels and different architectures are co-explored to identify the optimal balance between concurrency and resource usage.

Gerstlauer et al. [17] present six well-known ESL design methodologies and compare them using their proposed classification. Based on their classification, the design activities incorporated in ESL design methodologies often fit into an X-chart [17] that combines synthesis and quality assessment aspects of a design highlighted by two famous Y-charts, proposed by Gajski et al. [16] and Kienhuis et al. [27], respectively. ESL design methodologies mainly agree on the importance of separation of concerns for complexity management and follow the Y-chart [27] as a foundation for efficient high-level DSE. Following the Y-chart design approach that focuses on architecture exploration for a given behavioral model(s), all the presented design methodologies in [17], except DAEDALUS [46], lack "behavioral-level DSE" as called by Gerstlauer et al. [17]. DAEDALUS automatically converts behavioral specifications that are sequential C programs written in **Static Affine Nested Loop Programs** (**SANLPs**) style [50] to **Kahn Process Network** (**KPN**) [18] specifications using KPNgen tool [50]. KPNgen can also derive different input-output equivalent KPN specifications, for example, with different amounts of concurrency, using automated source-level transformations, which enables behavioral-level DSE. However, the requirement on the SANLP input for the KPNgen tool imposes a restriction on designers to write the behavioral specification in C programs in a special style. Moreover, automatic transformations are also possible only if this requirement is met since the transformations are defined based on the features of this style, for example, loop unrolling for statements enclosed in loops. In addition, this restriction requires

access to low-level programs that may not be accessible at the early stages of design. Our rule-based transformational design methodology eliminates this restriction. Instead, it enforces the designers to use a library of process constructors and data parallel skeletons to specify "legal" behaviors. This gives a great advantage, which is vital in a step-wise transformational design process (i.e., verification of each step); templates that are formal and verifiable. This also makes the definition of transformation rules more flexible; hence providing more opportunities for optimization transformations in comparison with KPNgen.

In a nutshell, the proposed design methodology causes a more complete design capture than previous ESL methodologies that only support architecture exploration (e.g., Metropolis [4], Koski [25], PeaCE/HOPES [20]) by providing support for application (transformation) space definition. We also propose a technique for automatic detection and enable automatic application of transformation rules that have been advocated by [38, 41], by carefully selecting the underlying models.

7 State of the Method: Formalisms and Tool Support

In this section, we examine the current state of the methodology in terms of formalism and tool support, as well as the generality of the methodology and the proposed techniques.

Application modeling framework: The concepts for application modeling are well-grounded in the functional programming paradigm, the theory of MoCs, and skeletons, and can be regarded as stable and mature. Any modeling language that supports these concepts can be effectively integrated into our proposed transformational design flow. We use ForSyDe framework, which has modeling libraries for various MoCs, supported by two modeling environments: ForSyDe-Shallow[9] and ForSyDe-Atom.[10]

RAMP views: The RAMP views make it possible to consider the interplay of transformations on different elements of RAMP. The concept of RAMPs is applied across all design steps that rely on the abstract graph representation: pattern matching, transformation space exploration, and DSE. ForSyDe IO[11] libraries are available to capture the intermediate representation of all four system views (RAMP views) as a unified common graph. These libraries are backed by a meta-modeling framework [21], which has been specifically extended in this work to support hierarchical process networks, the synchronous MoC, and data-parallel skeletons.

Automatic pattern matching: The proposed pattern matching technique is both general and language-agnostic, inheriting its language independence from the meta-modeling language used for the abstract graph representation, ForSyDe IO. The developed pattern matcher tool[12] performs pattern matching for all transformation rules specified in Listing 2. A similar approach to Algorithm 1 can be utilized to incorporate pattern identification for any transformation rule, requiring only the addition of the relevant trait hierarchy if it does not already exist.

Automatic transformations: The proposed technique can automate the application of any transformation rule. In the context of the abstract graph representation, a transformation is essentially a manipulation of the graph based on the format defined in the rule. The `Map-Split` transformation is automated, and other transformations are being automated using a similar approach to Algorithm 2 [3].

Design space exploration: Our transformation-based application space identification enables behavioral-level exploration. By applying different sequences of transformations, it generates

[9] https://github.com/forsyde/forsyde-shallow
[10] https://github.com/forsyde/forsyde-atom
[11] https://github.com/forsyde/forsyde-io
[12] https://github.com/forsyde/Transformation-Explorer

alternative application instances, which are then explored by IDeSyDe. IDeSyDe[13] is equipped with a genetic algorithm-based heuristic that effectively maps and schedules ForSyDe IO synchronous process networks onto tile-based platforms.

Tool integration: Both the Transformation Explorer and IDeSyDe use ForSyDe IO models as their input and output formats. Due to this shared data exchange format and the concept of views in ForSyDe IO, integrating these tools is straightforward. Currently, deriving abstract graph representations (ForSyDe IO models) from ForSyDe models requires manual effort. However, the Transformation Explorer is integrated with IDeSyDe.

8 Future Work

This section outlines future directions and offers recommendations for existing formalisms and tools that can support the proposed methodology.

Formal verification of transformations: Since transformations may change the behavior of the system, the requirements need to be verified after each transformation. As a customized verification approach for our methodology, we envision verified transformation rules. Verified transformations preserve the system's behavior correctness. To this aim, contract-based transformations will be the focus of future work, and in particular *assume-guarantee contracts* [5] are planned to be adopted. Assume-guarantee contracts are envisioned since they conform to transformation rules very well, aiming to guarantee certain application behavior after transforming the application model A to A', under assumptions on the structure of the original application model A.

Transformation rules: The transformation library needs to be extended. Although the transformation rules used in this article are examples of key categories of transformations for process networks, application-domain- and architecture-specific transformations can increase the benefits achieved by our transformational approach. Moreover, increasing the number of non-semantic preserving transformations improves model refinements.

As a practical tool to increase the number of available transformations, we consider allowing *untrusted transformations*, that is, transformations that are not yet validated. This also enables more freedom to inject the designers' knowledge into the design flow. The designer has the authority to insert such transformations (marked as untrusted) into the transformation library. The usage of untrusted transformations will require extra care and an additional verification effort of the final design as long as they are not validated. Once an untrusted transformation is validated by a suitable verification method, there transformation becomes trusted.

Non-semantic preserving transformations: There is no difference between matching the patterns of semantic preserving transformations and that of non-semantic preserving ones. However, their application is different. The implications of such non-semantic preserving transformations (i.e., arising from the changed semantics of the model) need to be considered while constructing the replacing system graph. At present, the suggested method for the application of transformations does not take the implications into account; therefore, a designer's supervision is needed for applying the non-semantic preserving transformations. The automatic application of such transformations is deferred to future work.

Parameterized transformations: The data parallelization transformations that extract the potential parallelization can be parameterized to define different degrees of parallelization. However, since the target platform is not necessarily preselected in our methodology, the range of parameters can not be predefined, which makes the application space definition more challenging. An exploration tool that jointly explores the application and platform space can choose the degree of

[13] https://github.com/forsyde/IDeSyDe

parallelization that makes the best trade-off between performance metrics (e.g., throughput, the number of cores, etc).

Efficient exploration of transformation space: The results indicate that the transformation space can be huge, even for small systems. To efficiently navigate this space, the Transformation Explorer can be supported by the DSE tool. Utilizing feedback from the DSE tool, which provides performance estimations, the Transformation Explorer can selectively focus on transforming specific parts of the system graph that are deemed worthwhile for transformation.

9 Conclusion

The article has extended existing transformational design approaches into a novel system design methodology that addresses the complexity of embedded system design and enhances design feasibility and optimality. The methodology enables high-level transformations that have more impact on design efficiency. The key idea is to center the methodology around a coherent system model (RAMP) and its underlying unified abstract graph representation, where the system at each transformation step is expressed as a tuple of requirements, application model, platform model, and mapping model. By this, the effect of transformations on different system properties is considered, and the design capture is improved, which significantly increases the potential to identify an efficient implementation.

A tool-automated pattern-matching technique, built on foundations of graphs and trait hierarchies, is introduced for detecting all possible transformations, and the groundwork for automated transformations has been laid. Leveraging this foundation, a tool for automatic transformations is currently in development (Section 8 delves into the current state of the methodology and outlines directions for future research).

The key concepts and mechanisms of the proposed methodology have been demonstrated through a representative example. We further evaluated the methodology by applying it to the design of both an image processing system and a cloud detection system. The results show a clear improvement in the performance metrics of the transformed system models compared to those of the original models.

References

[1] Ralph-Johan Back and Joakim Wright. 2012. *Refinement Calculus: A Systematic Introduction*. Springer Science & Business Media.

[2] John Backus. 1978. Can programming be liberated from the von Neumann style? A functional style and its algebra of programs. *Communications of the ACM* 21, 8 (1978), 613–641.

[3] Fahimeh Bahrami, Rodolfo Jordão, Ingo Sander, and George Ungureanu. 2024. Automatic parallelization of embedded software via hierarchical process network transformations. In *Proceedings of the 2024 Forum on Specification & Design Languages*. IEEE, 1–9.

[4] Felice Balarin, Yosinori Watanabe, Harry Hsieh, Luciano Lavagno, Claudio Passerone, and Alberto Sangiovanni-Vincentelli. 2003. Metropolis: An integrated electronic system design environment. *Computer* 36, 4 (2003), 45–52.

[5] Albert Benveniste, Benoît Caillaud, Alberto Ferrari, Leonardo Mangeruca, Roberto Passerone, and Christos Sofronis. 2008. Multiple viewpoint contract-based specification and design. In *Proceedings of the Formal Methods for Components and Objects: 6th International Symposium, FMCO 2007, Amsterdam, The Netherlands, October 24–26, 2007, Revised Lectures 6*. Springer, 200–225.

[6] Tobias Blickle, Jürgen Teich, and Lothar Thiele. 1998. System-level synthesis using evolutionary algorithms. *Design Automation for Embedded Systems* 3, 1 (1998), 23–58.

[7] Manfred Broy and Ketil Stølen. 2012. *Specification and Development of Interactive Systems: Focus on Streams, Interfaces, and Refinement*. Springer Science & Business Media.

[8] Murray Cole. 1989. *Algorithmic Skeletons: Structured Management of Parallel Computation*. Pitman, London.

[9] Patrick Cousot and Radhia Cousot. 2002. Systematic design of program transformation frameworks by abstract interpretation. In *Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. 178–190.

[10] David Culler, Jaswinder Pal Singh, and Anoop Gupta. [n. d.]. *Parallel Computer Architecture: A Hardware/Software Approach*. Gulf Professional Publishing. googlebooks:MHfHC4Wf3K0C
[11] Erwin A. de Kock. 2002. Multiprocessor mapping of process networks: A JPEG decoding case study. In *Proceedings of the 15th International Symposium on System Synthesis*. 68–73.
[12] Reinhard Diestel. 2010. *Graph Theory*. Springer, Heidelberg New York.
[13] Edsger W. Dijkstra. 1968. A constructive approach to the problem of program correctness. *BIT Numerical Mathematics* 8, 3 (1968), 174–186.
[14] Rolf Ernst. 1998. Codesign of embedded systems: Status and trends. *IEEE Design & Test of Computers* 15, 2 (1998), 45–54.
[15] Alberto Ferrari and Alberto Sangiovanni-Vincentelli. 1999. System design: Traditional concepts and new paradigms. In *Proceedings of the 1999 IEEE International Conference on Computer Design: VLSI in Computers and Processors (Cat. No. 99CB37040)*. IEEE, 2–12.
[16] Daniel D. Gajski, Samar Abdi, Andreas Gerstlauer, and Gunar Schirner. 2009. *Embedded System Design: Modeling, Synthesis and Verification*. Springer Science & Business Media.
[17] Andreas Gerstlauer, Christian Haubelt, Andy D. Pimentel, Todor P. Stefanov, Daniel D. Gajski, and Jürgen Teich. 2009. Electronic system-level synthesis methodologies. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 28, 10 (2009), 1517–1530.
[18] G. Kahn. 1974. The semantics of a simple language for parallel programming. In *Proceedings of the IFIP Congress 74 (Amsterdam, The Netherlands)*. 471–475.
[19] Jeffrey O. Grady. 2010. *System Requirements Analysis*. Elsevier.
[20] Soonhoi Ha, Choonseung Lee, Youngmin Yi, Seongnam Kwon, and Young-Pyo Joo. 2006. Hardware-software codesign of multimedia embedded systems: The peace. In *Proceedings of the 12th IEEE International Conference on Embedded and Real-Time Computing Systems and Applications*. IEEE, 207–214.
[21] Rodolfo Jordão, Fahimeh Bahrami, Rui Chen, and Ingo Sander. 2022. A multi-view and programming language agnostic framework for model-driven engineering. In *Proceedings of the Forum on Specification and Design Languages, SEP 14–16, 2022, Linz, AUSTRIA*. Institute of Electrical and Electronics Engineers (IEEE).
[22] Rodolfo Jordão, Fahimeh Bahrami, Yu Yang, Matthias Becker, Ingo Sander, and Kathrin Rosvall. 2024. Multi-objective preference-free exact design space exploration of static DSP on multicore platforms. In *Proceedings of the 2024 Forum on Specification & Design Languages*. IEEE, 1–9.
[23] R. Jordão, M. Becker, and I. Sander. 2024. IDeSyDe: Systematic design space exploration via design space identification. *ACM Transactions on Design Automation of Electronic Systems* 29, 5 (2024), 1–45.
[24] Andrew B. Kahng, Bin Li, Li-Shiuan Peh, and Kambiz Samadi. 2009. ORION 2.0: A fast and accurate NoC power and area model for early-stage design space exploration. In *Proceedings of the 2009 Design, Automation & Test in Europe Conference & Exhibition*. IEEE, 423–428.
[25] Tero Kangas, Petri Kukkala, Heikki Orsila, Erno Salminen, Marko Hännikäinen, Timo D. Hämäläinen, Jouni Riihimäki, and Kimmo Kuusilinna. 2006. UML-based multiprocessor SoC design framework. *ACM Transactions on Embedded Computing Systems* 5, 2 (2006), 281–320.
[26] Kurt Keutzer, A. Richard Newton, Jan M. Rabaey, and Alberto Sangiovanni-Vincentelli. 2000. System-level design: Orthogonalization of concerns and platform-based design. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 19, 12 (2000), 1523–1543.
[27] B. Kienhuis, F. Deprettere, P. van der Wolf, and K. Vissers. 2022. A methodology to design programmble embedded systems: The y-chart approach. *Embedded Processor Design Challenges, Ser.* Lecture Notes in Computer Science, Berlin, Germany: Springer-Verlag, 2002, 2268 (2022), 18–37.
[28] Edward Ashford Lee and David G. Messerschmitt. 1987. Static scheduling of synchronous data flow programs for digital signal processing. *IEEE Transactions on Computers* 100, 1 (1987), 24–35.
[29] Edward A. Lee and Alberto Sangiovanni-Vincentelli. 1998. A framework for comparing models of computation. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 17, 12 (1998), 1217–1229.
[30] Sjoerd Meijer, Bart Kienhuis, Alex Turjan, and Erwin de Kock. 2007. A process splitting transformation for Kahn process networks. In *Proceedings of the 2007 Design, Automation & Test in Europe Conference & Exhibition*. IEEE, 1–6.
[31] Sjoerd Meijer, Hristo Nikolov, and Todor Stefanov. 2009. On compile-time evaluation of process partitioning transformations for Kahn process networks. In *Proceedings of the 7th IEEE/ACM International Conference on Hardware/software Codesign and System Synthesis*. 31–40.
[32] Sjoerd Meijer, Hristo Nikolov, and Todor Stefanov. 2010. Throughput modeling to evaluate process merging transformations in polyhedral process networks. In *Proceedings of the 2010 Design, Automation & Test in Europe Conference & Exhibition*. IEEE, 747–752.
[33] Pierluigi Nuzzo, Alberto L. Sangiovanni-Vincentelli, Davide Bresolin, Luca Geretti, and Tiziano Villa. 2015. A platform-based design methodology with contracts and related tools for the design of cyber-physical systems. *Proceedings of the IEEE* 103, 11 (2015), 2104–2132.

[34] Helmuth Partsch and Ralf Steinbrüggen. 1983. Program transformation systems. *ACM Computing Surveys (CSUR)* 15, 3 (1983), 199–236.
[35] Alberto Pettorossi and Maurizio Proietti. 1996. Rules and strategies for transforming functional and logic programs. *ACM Computing Surveys (CSUR)* 28, 2 (1996), 360–414.
[36] Steffen Prochnow and Reinhard von Hanxleden. 2007. The use of complex stateflow-charts with KIEL–an automotive case study. In *Proceedings of 5th GI-Workshop Automotive Software Engineering (ASE'07)*. Bremen, Germany.
[37] Tarvo Raudvere, Ingo Sander, and Axel Jantsch. 2008. Application and verification of local nonsemantic-preserving transformations in system design. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 27, 6 (2008), 1091–1103.
[38] Hideki John Reekie. 1995. *Realtime signal processing*. Ph. D. Dissertation. University of Technology at Sydney, Australia.
[39] John Reekie and John Potter. 1993. Process network transformation. In *Parallel Computing and Transputers (PCAT-93)*, David Arnold (Eds.). IOS Press, 376–383.
[40] Ingo Sander. 2003. *System modeling and design refinement in ForSyDe*. Ph. D. Dissertation. KTH Royal Institute of Technology.
[41] Ingo Sander and Axel Jantsch. 2004. System modeling and transformational design refinement in ForSyDe. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 23, 1 (2004), 17–32.
[42] C. Schwartz, I. Sander, R. Jordão, F. Bruhn, M. Persson, J. Ekblad, and C. Fuglesang. 2022. On-board satellite data processing to achieve smart information collection. In *Optics, Photonics and Digital Technologies for Imaging Applications VII*, P. Schelkens and T. Kozacki (Eds.). vol. 12138, International Society for Optics and Photonics. SPIE, 2022. 121380I. [Online]. Available: https://doi.org/10.1117/12.2620955
[43] Joseph Sifakis. 2015. System design automation: Challenges and limitations. *Proceedings of the IEEE* 103, 11 (2015), 2093–2103.
[44] David B. Skillicorn. 1994. *Foundations of Parallel Programming*. Cambridge University Press.
[45] Todor Stefanov, Bart Kienhuis, and Ed Deprettere. 2002. Algorithmic transformation techniques for efficient exploration of alternative application instances. In *Proceedings of the 10th International Symposium on Hardware/software Codesign*. 7–12.
[46] Todor Stefanov, Andy Pimentel, and Hristo Nikolov. 2017. DAEDALUS: System-level design methodology for streaming multiprocessor embedded systems on chips. In *Handbook of Hardware/Software Codesign*, Soonhoi Ha and Jürgen Teich (Eds.). Springer Netherlands, Dordrecht, 983–1018.
[47] Alexander Stegmeier, Martin Frieb, Ralf Jahr, and Theo Ungerer. 2015. Algorithmic skeletons for parallelization of embedded real-time systems. In *Proceedings of the 3rd Workshop on High-Performance and Real-time Embedded Systems*.
[48] George Ungureanu. 2021. *ForSyDe-Atom: Design of Heterogeneous Embedded Systems – Taming Complexity with Layers, Atoms and Patterns*. Ph. D. Dissertation. KTH Royal Institute of Technology. Retrieved from http://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-303934
[49] Jonne Van Wijngaarden and Eelco Visser. 2003. Program transformation mechanics. a classification of mechanisms for program transformation with a survey of existing transformation systems. (Technical report, UU-CS, May 2003).
[50] S. Verdoolaege, H. Nikolov, and T. Stefanov. 2007. PN: a tool for improved derivation of process networks. *EURASIP Journal on Embedded Systems* 2007 (2007).
[51] Eelco Visser. 2005. A survey of strategies in rule-based program transformation systems. *Journal of Symbolic Computation* 40, 1 (2005), 831–873.

Received 1 February 2024; revised 20 October 2024; accepted 9 January 2025